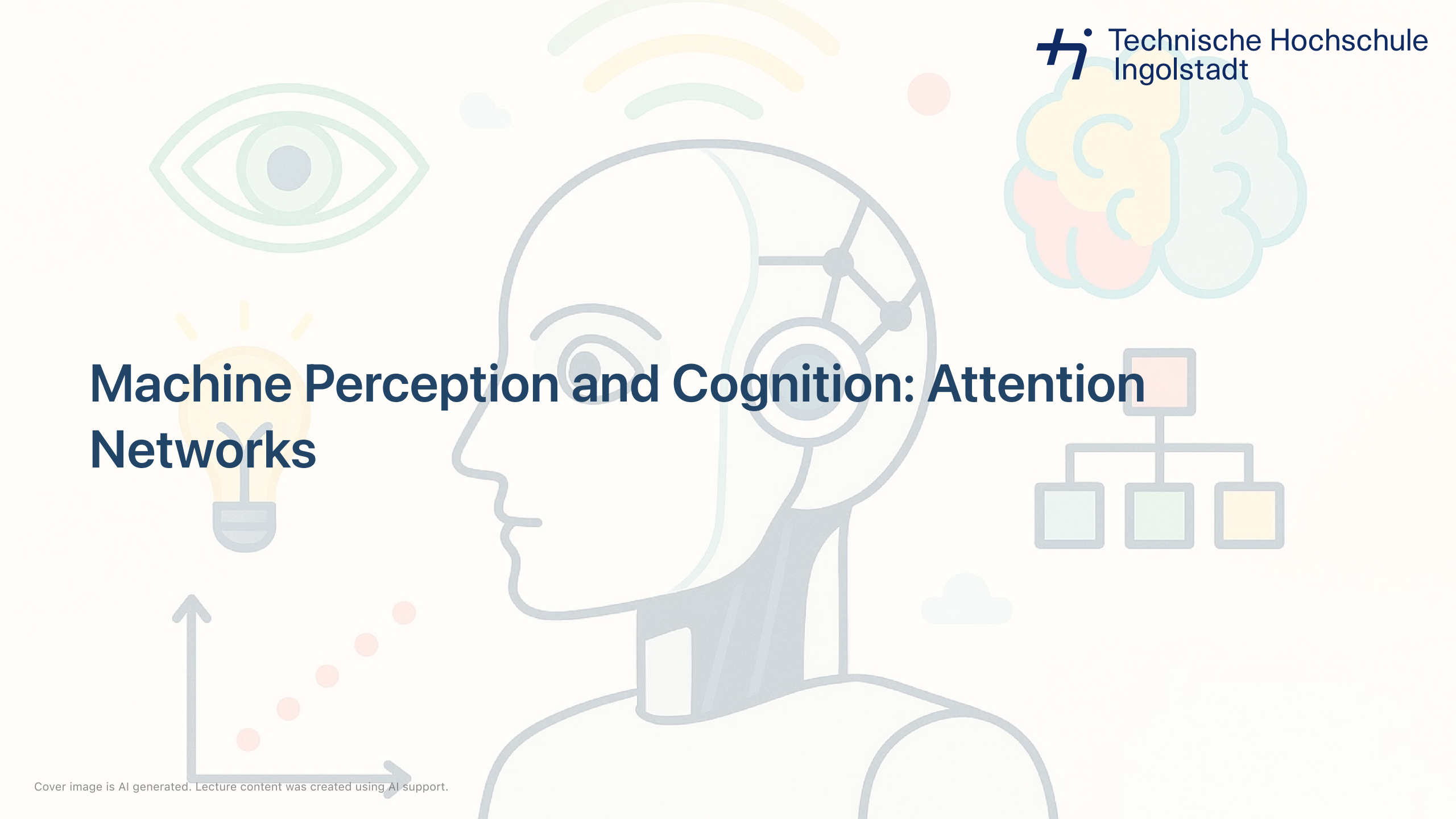
Technische Hochschule Ingolstadt



Machine Perception and Cognition: Attention Networks

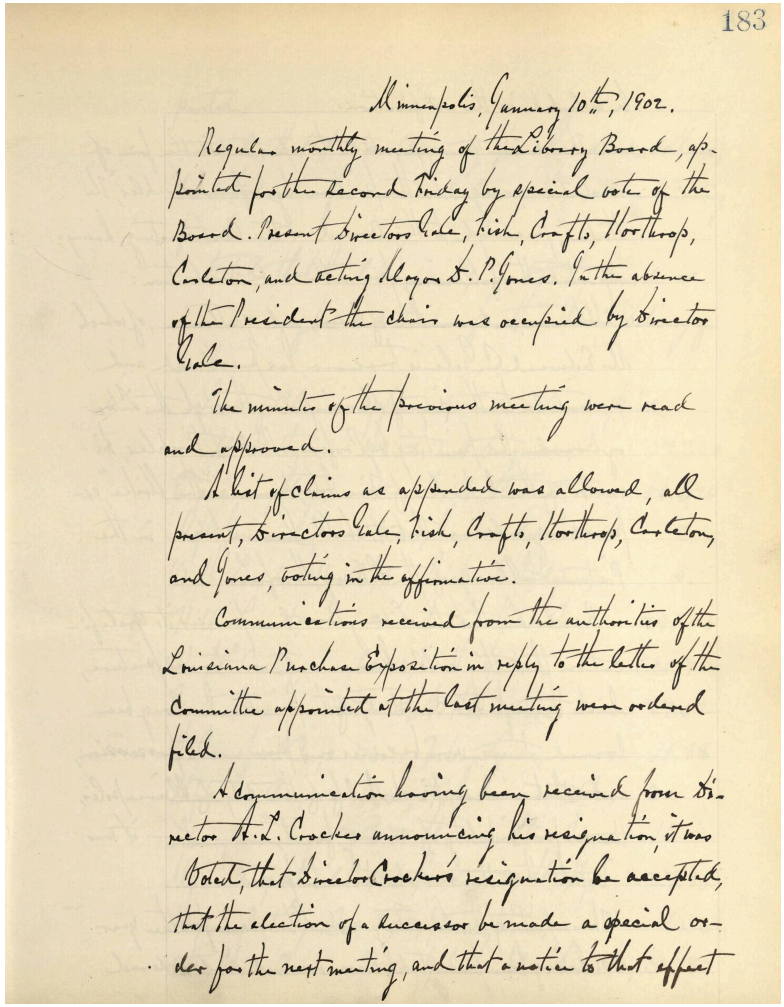
Cover image is AI generated. Lecture content was created using AI support.

From CTC to Attention

In the CTC-based handwriting-recognition approach discussed earlier, CTC aligned a **1D input sequence** with an output character sequence.

This works because line images can be represented as left-to-right sequences. Now consider a full handwritten page, where characters are arranged across multiple lines in a two-dimensional layout:

Applying CTC to this?



183

Minneapolis, January 10th, 1902.

Regular monthly meeting of the Library Board, appointed for the second Friday by special vote of the Board. Present Directors Gale, Fish, Crafts, Northrop, Coolidge, and acting Mayor D. P. Jones. In the absence of the President the chair was occupied by Director Gale.

The minutes of the previous meeting were read and approved.

A list of claims as appended was allowed, all present, Directors Gale, Fish, Crafts, Northrop, Coolidge and Jones, voting in the affirmative.

Communications received from the authorities of the Louisiana Purchase Exposition in reply to the letter of the Committee appointed at the last meeting were ordered filed.

A communication having been received from Director H. L. Crocker announcing his resignation, it was

Voted, that Director Crocker's resignation be accepted, that the election of a successor be made a special order for the next meeting, and that a notice to that effect

Image source: Meeting Minutes, Minneapolis Public Library Board of Directors for 1902, Public domain, via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Meeting_Minutes,_Minneapolis_Public_Library_Board_of_Directors_for_1902_-_DPLA_-_f5bb3cdbb77d3ee0bf0daa93269c9c2f_(page_1).jpg

Why CTC Falls Short

CTC (Connectionist Temporal Classification) works well for **line-level** handwriting recognition, but a full handwritten page poses new challenges:

- **Unknown 2D layout:** CTC assumes a single, ordered 1D sequence — a paragraph has multiple lines in a two-dimensional arrangement
- **Variable reading order:** lines must be found and traversed, which CTC cannot determine on its own
- **Long-range context:** recognizing a word may require context from a distant region of the page

What we need is a mechanism that can **selectively focus** on the relevant part of an input, with the focus controlled by the current decoding state.

→ This is exactly what **attention networks** provide.

Attention in Psychology

From Wikipedia, page "Attention":

> **Attention** is the concentration of awareness directed at some task or phenomenon while mostly excluding others.

Attention in Neural Networks

So far we have discussed:

- fully connected networks, which process all data at once.
- convolutional neural networks, which process data within a sliding window.
- recurrent neural networks, which process data in a sequential order.

And now:

- attention networks, which route information by **content-dependent selection**.

Attention in Neural Networks

CNN, RNN and self-attention neural network:

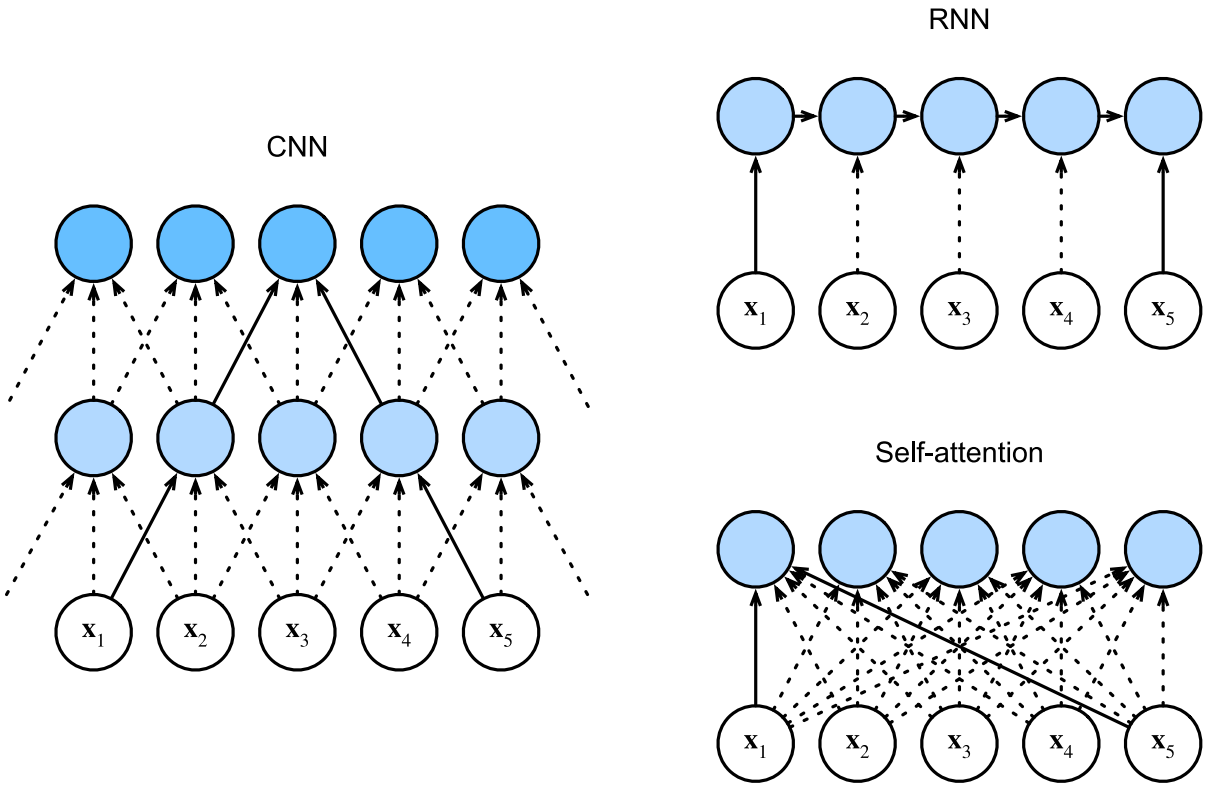


Image source: Zhang, Aston and Lipton, Zachary C. and Li, Mu and Smola, Alexander J., CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Self-attention_in_CNN,_RNN,_and_self-attention.svg

Attention in Neural Networks

Terminology similar to a database lookup:

- *Query* specifies what to look for and process next.
- *Keys* are the set of items that can be matched against the query.
- *Values* are the set of items attached to the keys, defining the data to be processed.

Attention in Neural Networks

General overview with query, keys, values and key-value pairs as memory:

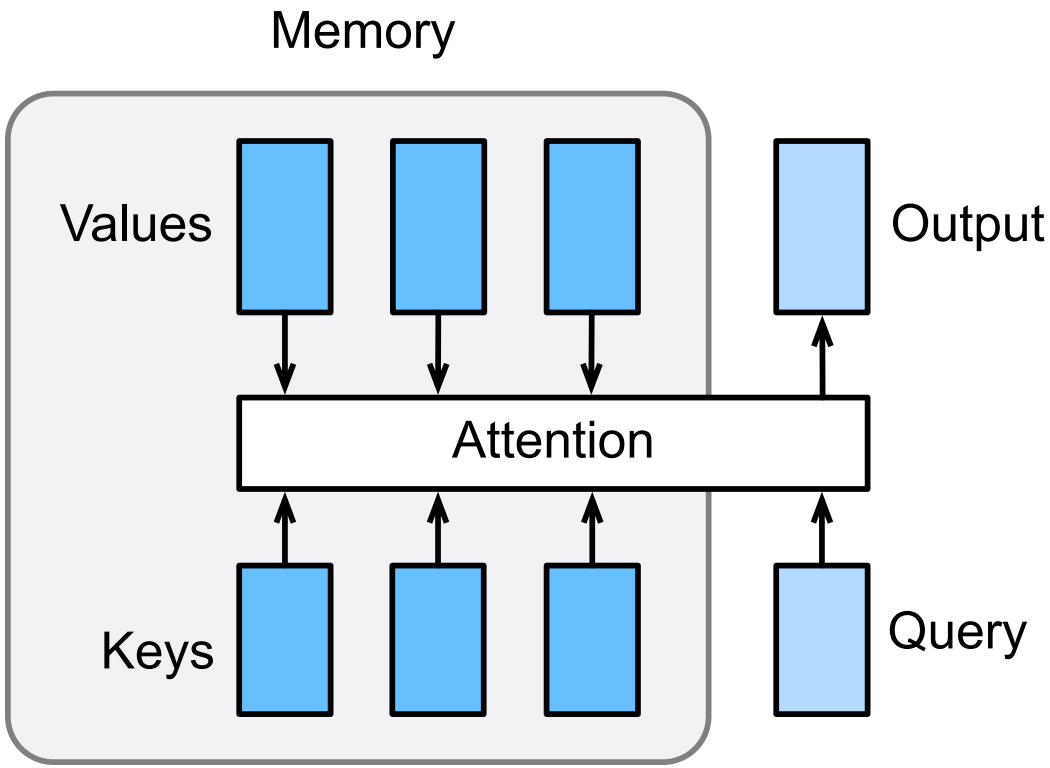


Image source: Zhang, Aston and Lipton, Zachary C. and Li, Mu and Smola, Alexander J., CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Attention_mechanism_overview.svg

Attention in Neural Networks

Detail overview with a query, keys, values and softmax scoring:

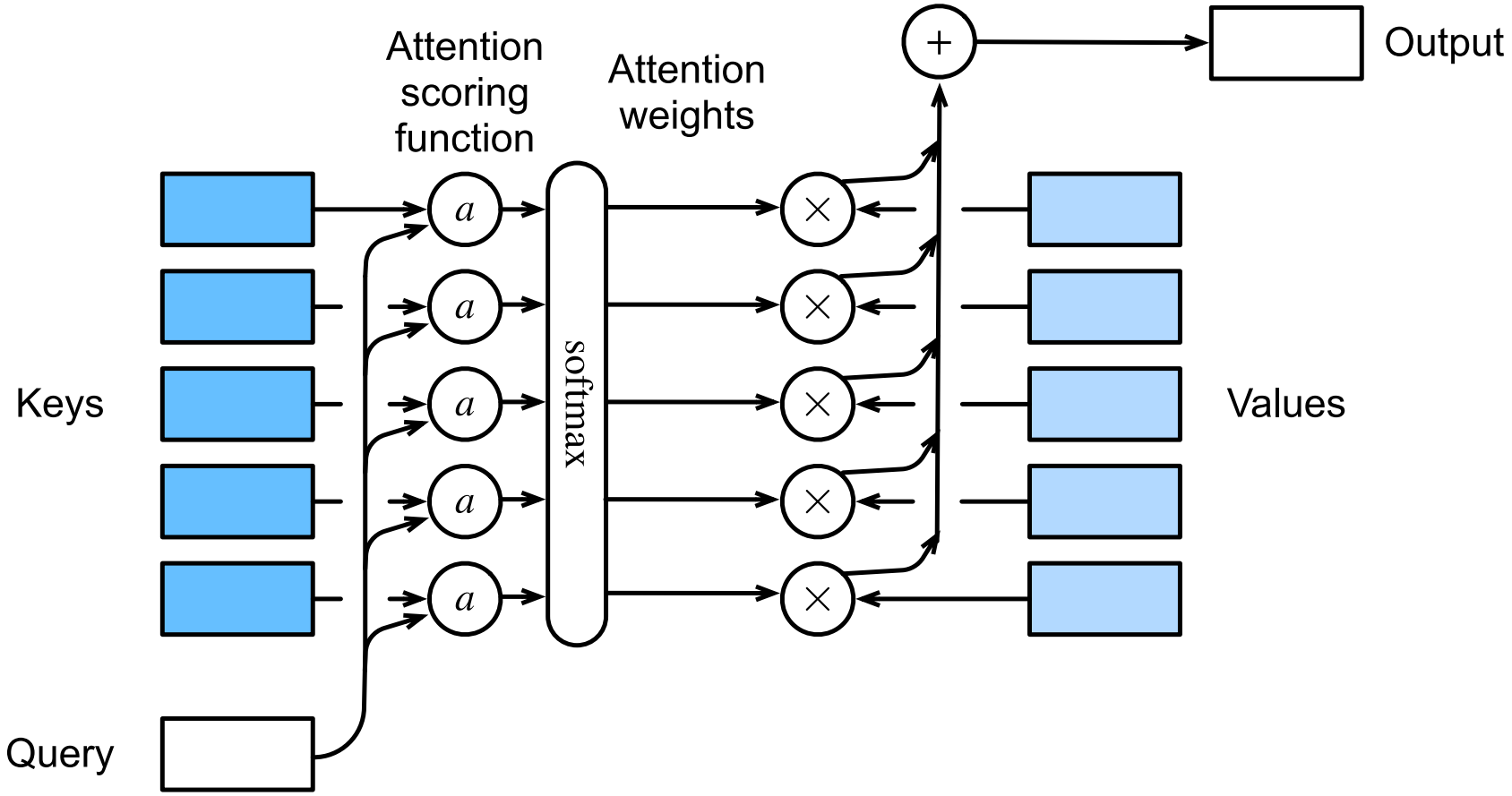


Image source: Zhang, Aston and Lipton, Zachary C. and Li, Mu and Smola, Alexander J., CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Attention_mechanism_output.svg

Scoring Functions: Overview

How do we measure the match between a *query* $\mathbf{q}$ and a *key* $\mathbf{k}$?

A **scoring function** $a(\mathbf{q}, \mathbf{k}) \in \mathbb{R}$ assigns a scalar similarity value to each query–key pair.

Common choices:

- **Additive attention** (Bahdanau et al., 2015): learned compatibility via a single hidden layer
- **Dot-product attention** (Luong et al., 2015): inner product — fast and parameter-free
- **Scaled dot-product** (Vaswani et al., 2017): dot-product with a $1/\sqrt{d_k}$ factor for stability

In all cases, scores are converted to a **probability distribution** via softmax and then used to compute a weighted sum of values.

Scoring Functions: Additive Attention

Additive (Bahdanau) attention uses a one-hidden-layer MLP with learnable parameters:

$$a(\mathbf{q}, \mathbf{k}) = \mathbf{w}_v^\top \tanh(\mathbf{W}_q\mathbf{q} + \mathbf{W}_k\mathbf{k})$$

- $\mathbf{W}_q \in \mathbb{R}^{h \times d_q}$, $\mathbf{W}_k \in \mathbb{R}^{h \times d_k}$: projection matrices mapping query and key to a common hidden space of size h
- $\mathbf{w}_v \in \mathbb{R}^h$: output weight vector

Properties:

- Can compare vectors of **different dimensionality** ($d_q \neq d_k$)
- Introduces **learnable parameters** in the scoring step
- Originally proposed for machine translation with encoder–decoder RNNs (2015)
- Their extension adds location features from the previous attention weights

Scoring Functions: Dot-Product Attention

Dot-product (Luong) attention computes the inner product directly:

$$a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^\top \mathbf{k}$$

Requires query and key to have the **same dimensionality** d_k.

The complete attention output for one query and m key-value pairs:

$$f(\mathbf{q}, (\mathbf{k}_1, \mathbf{v}_1), \ldots, (\mathbf{k}_m, \mathbf{v}_m)) = \sum_{i=1}^{m} \frac{\exp(\mathbf{q}^\top \mathbf{k}_i)}{\sum_{j=1}^{m} \exp(\mathbf{q}^\top \mathbf{k}_j)} \mathbf{v}_i$$

- **No additional parameters** — score directly measures geometric alignment
- Computationally very efficient on modern hardware

Scoring Functions: Scaled Dot-Product

Scaled dot-product attention (Vaswani et al., 2017):

$$\mathbf{A}(\mathbf{Q}, \mathbf{K}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)$$

where:

- $\mathbf{Q} \in \mathbb{R}^{n \times d_k}$ — matrix of n query vectors
- $\mathbf{K} \in \mathbb{R}^{m \times d_k}$ — matrix of m key vectors

This formulation processes all n queries **simultaneously** in matrix form, enabling efficient batch computation.

Scoring Functions: From Scores to Output

Full matrix-form computation for n queries and m key-value pairs:

Step 1 — Compute the score matrix:

$$\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \in \mathbb{R}^{n \times m}$$

Step 2 — Apply softmax row-wise (normalize over keys):

$$\mathbf{A} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{n \times m}, \qquad \sum_{j=1}^{m} a_{ij} = 1 \; \forall \, i$$

Step 3 — Aggregate values:

$$\mathbf{O} = \mathbf{A}\mathbf{V} \in \mathbb{R}^{n \times d_v}$$

Scoring Functions: From Scores to Output

Each output row $\mathbf{O}_i$ is a **context vector**: a convex combination of values, weighted by how closely query i matches each key.

Scoring Functions: Comparison

The table counts only **scoring-function parameters**. Learned Q, K, V projections are part of a complete attention layer, but are counted separately from scoring-function parameters.

Property	Additive	Dot-Product	Scaled Dot-Product
Scoring-function parameters	Yes ($\mathbf{W}_q, \mathbf{W}_k, \mathbf{w}_v$)	No	No
Different dims ($d_q \neq d_k$)?	✓	×	×
Computational cost	Higher	Low	Low
Gradient stability	Good	May saturate	Good
Used in	RNN attention (2015)	Early NMT	Modern attention (2017+)

Self-Attention: Introduction

In **self-attention**, the sequence attends to **itself** — queries, keys, and values all originate from the same input sequence.

Given input $\mathbf{X} \in \mathbb{R}^{n \times d}$ (a sequence of n tokens with embedding dimension d):

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V$$

where $\mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d \times d_k}$ and $\mathbf{W}^V \in \mathbb{R}^{d \times d_v}$ are **learned projection matrices**.

Each position can directly attend to **any other position** — enabling global context awareness without sequential processing.

Self-Attention: Why Learn Q, K, V Projections?

The learned projections allow each role to capture a **different aspect** of the same token:

- $\mathbf{W}^Q$ projects to "what am I looking for?"
- $\mathbf{W}^K$ projects to "what do I offer to be matched against?"
- $\mathbf{W}^V$ projects to "what information do I contribute?"

This gives the model **flexible representational power**: the query and key spaces can be optimized for accurate matching, while the value space can be optimized for informative aggregation.

Self-Attention: Step-by-Step Computation

Given $\mathbf{X} \in \mathbb{R}^{n \times d}$:

1. **Project** to queries, keys, values:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{Q}, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^{K}, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^{V}$$

2. **Score** all position pairs:

$$\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_k}} \in \mathbb{R}^{n \times n}$$

3. **Normalize** with softmax:

$$\mathbf{A} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{n \times n}$$

Self-Attention: Step-by-Step Computation

4. Aggregate values:

$$\mathbf{O} = \mathbf{A}\mathbf{V} \in \mathbb{R}^{n \times d_v}$$

The output $\mathbf{O}_i$ is a weighted sum of all values, with weights determined by how closely position i matches every other position.

Self-Attention: Attention Weight Matrix

The $n \times n$ matrix $\mathbf{A}$ encodes all pairwise relationships in the sequence:

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix}$$

- a_{ij}: how much position i **attends to** position j
- Each row sums to 1 (softmax normalization)
- Diagonal: position attends to itself
- Off-diagonal: cross-position attention

Self-Attention: Example in NLP

*"The animal didn't cross the street because **it** was too tired."*

What does "it" refer to? → "animal" (not "street")

Self-attention can learn: $a_{\text{it, animal}} \gg a_{\text{it, street}}$

Contrast with RNNs: information from "animal" must propagate through every intermediate token before reaching "it" — subject to vanishing gradients and information loss.

Self-attention enables **global context awareness** at every position simultaneously, regardless of distance in the sequence.

Self-Attention: Masked Self-Attention

For **autoregressive** generation, position i may only attend to positions $\leq i$ — future tokens must be hidden.

Causal mask: Add $-\infty$ to future positions before softmax:

$$M_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right)$$

After softmax, $-\infty$ entries produce attention weights of exactly 0.

Self-Attention: Masked Self-Attention

Application: Language models (GPT family) use masked self-attention to support left-to-right training with **teacher forcing**: all positions are trained in parallel during training, while generation proceeds one token at a time at inference.

Cross-Attention: Historical Context (Bahdanau, 2015)

Cross-attention was introduced **before Transformers** to overcome the fixed-length bottleneck of encoder–decoder RNNs.

Bahdanau et al. (2015) — Neural Machine Translation:

- **Encoder:** Bidirectional RNN produces hidden states $\mathbf{h}_1, \ldots, \mathbf{h}_m$
- **Previous decoder state** before step t: $\mathbf{s}_{t-1}$
- **Additive score:** $e_{tj} = \mathbf{w}_v^\top \tanh(\mathbf{W}_s \mathbf{s}_{t-1} + \mathbf{W}_h \mathbf{h}_j)$
- **Context vector:** $\mathbf{c}_t = \sum_j \alpha_{tj} \mathbf{h}_j$, where $\boldsymbol{\alpha}_t = \mathrm{softmax}(\mathbf{e}_t)$

This was the first successful attention mechanism for NLP — it allowed the decoder to dynamically focus on different parts of the source sentence, achieving major improvements in translation quality.

Cross-Attention: Introduction

In **cross-attention**, queries come from one sequence while keys and values come from a **different** sequence.

- $\mathbf{Q} = \mathbf{X}^{(1)}\mathbf{W}^{Q}$ — from the *target* sequence of length n
- $\mathbf{K} = \mathbf{X}^{(2)}\mathbf{W}^{K},\ \mathbf{V} = \mathbf{X}^{(2)}\mathbf{W}^{V}$ — from the *source* sequence of length m

The computation is identical to scaled dot-product attention:

$$\mathrm{CrossAttn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_k}}\right)\mathbf{V} \in \mathbb{R}^{n \times d_v}$$

Each position in the target sequence **selectively attends** to the most relevant positions in the source sequence.

Cross-Attention: Encoder–Decoder Setting

Cross-attention is the **bridge** between encoder and decoder in sequence-to-sequence models:

Encoder processes the source sequence $\mathbf{X}^{(\text{src})}$ → produces key and value representations $\mathbf{K}_{\text{enc}}, \mathbf{V}_{\text{enc}}$

Decoder generates the target sequence token by token → provides query representations $\mathbf{Q}_{\text{dec}}$

Cross-Attention: Encoder–Decoder Setting

At each decoder step t, the context vector is:

$$\mathbf{c}_t = \mathrm{softmax}\left(\frac{\mathbf{q}_t \, \mathbf{K}_{\mathrm{enc}}^{\top}}{\sqrt{d_k}}\right)\mathbf{V}_{\mathrm{enc}}$$

- $\mathbf{c}_t$: a dynamic summary of the source, tailored to what the decoder currently needs
- Attention weights visualize which source positions the decoder is "reading"

Cross-Attention: Applications

Cross-attention enables one sequence to **condition on** another — the key building block for multi-modal models:

Task	Query source	Key / Value source
Machine Translation	Target tokens (decoder)	Source tokens (encoder)
Image Captioning	Caption tokens	Image patch features
Visual Question Answering	Question tokens	Image features
Speech Recognition	Transcript (decoder)	Audio features (encoder)

In all cases, the same mathematical mechanism allows flexible, learned alignment between two different domains.

Cross-Attention vs. Self-Attention

Property	Self-Attention	Cross-Attention
Q source	Same sequence	Target sequence
K, V source	Same sequence	Source sequence
Attention matrix size	$n \times n$	$n \times m$
Typical location	Encoder	Encoder–Decoder bridge
Models relationships	Within a sequence	Between two sequences

Both mechanisms use **identical scaled dot-product** computations — only the origin of Q, K, V differs.

Multi-Head Attention: Motivation

A single attention head computes **one weighted combination** of values per position — one learned attention pattern.

But natural language (and other sequences) involves **multiple types of relationships simultaneously**:

- **Syntactic:** subject–verb agreement, object dependencies
- **Semantic:** coreference, synonymy
- **Local:** phrase-level structure (adjacent words)
- **Long-range:** discourse relations across sentences

Multi-Head Attention: Motivation

A single attention pattern cannot specialize in all of these at once.

Idea: Run H attention operations in **parallel**, each with independent learned projections, then combine their outputs.

Multi-Head Attention: Architecture

For H attention heads, each head $h = 1, \ldots, H$ independently computes:

$$\text{head}_h = \text{Attention}(\mathbf{Q}\mathbf{W}_h^Q,\ \mathbf{K}\mathbf{W}_h^K,\ \mathbf{V}\mathbf{W}_h^V)$$

with per-head projection matrices:
$\mathbf{W}_h^Q, \mathbf{W}_h^K \in \mathbb{R}^{d \times d_k}$ and $\mathbf{W}_h^V \in \mathbb{R}^{d \times d_v}$.

All heads are **concatenated** and linearly projected back to dimension d:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)\, \mathbf{W}^O$$

where $\mathbf{W}^O \in \mathbb{R}^{Hd_v \times d}$ is the output projection matrix.

Multi-Head Attention: What Different Heads Learn

Empirical studies of trained multi-head attention models reveal that heads **specialize** in distinct relationship types:

Clark et al. (2019), "What Does BERT Look At? An Analysis of BERT's Attention" showed that individual heads in BERT are interpretable and capture linguistically meaningful patterns.

Multi-Head Attention: Benefits

Multi-head attention provides several advantages over single-head attention:

1. **Diverse patterns:** Each head independently learns a different attention distribution
2. **Subspace specialization:** Heads operate in separate low-dimensional subspaces of Q, K, V
3. **Ensemble-like robustness:** Multiple independent computations reduce variance
4. **Efficient parallelism:** All heads are computed simultaneously on modern hardware
5. **Scalable capacity:** More heads add distinct subspaces. If the total model dimension is fixed, smaller per-head dimensions keep the parameter count roughly constant

Multi-head attention is the central building block of virtually every modern large model across NLP, computer vision, and speech processing.

Attention in Practice: Handwriting Recognition

We now return to the question from the opening slide.

Recall: CTC alone cannot handle full handwritten paragraphs. Attention provides the missing piece:

- The model can **dynamically select** which region of the image to process next
- It learns to **follow the reading order** from the data, without hard-coded segmentation rules
- Each output character is conditioned on a **soft alignment** over the image features

The following slides show two landmark papers that apply these ideas to handwritten paragraph recognition — built on the attention techniques from this lecture.

What is MDLSTM?

Multi-Dimensional LSTM (MDLSTM) extends the standard LSTM to process 2D grids such as images:

- A standard LSTM reads a sequence left-to-right, carrying one hidden state forward
- An MDLSTM scans the image with four corner-to-corner directional traversals, one starting from each image corner
- Each pixel location receives context from neighbouring positions already visited in that pass
- Multiple directional passes are combined so every feature map cell captures broad 2D context

Scan, Attend & Read: Normal MDLSTM Network

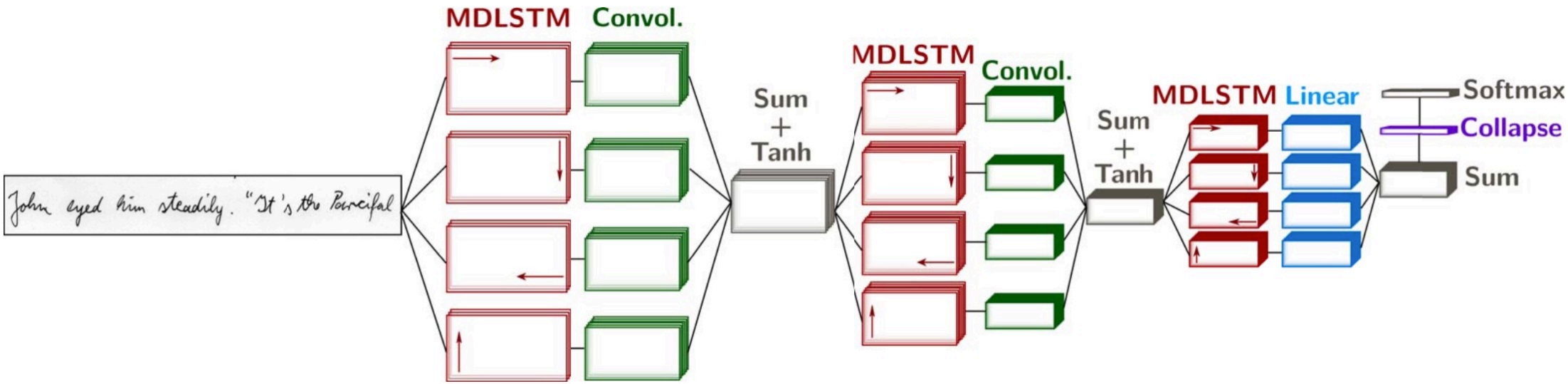


Image source: Bluche, T., Louradour, J., & Messina, R. (2017, November). Scan, attend and read: End-to-end handwritten paragraph recognition with MDLSTM attention. In 2017 14th IAPR international conference on document analysis and recognition (ICDAR) (Vol. 1, pp. 1050-1055). IEEE.

Scan, Attend & Read: Attention and MDLSTM Network

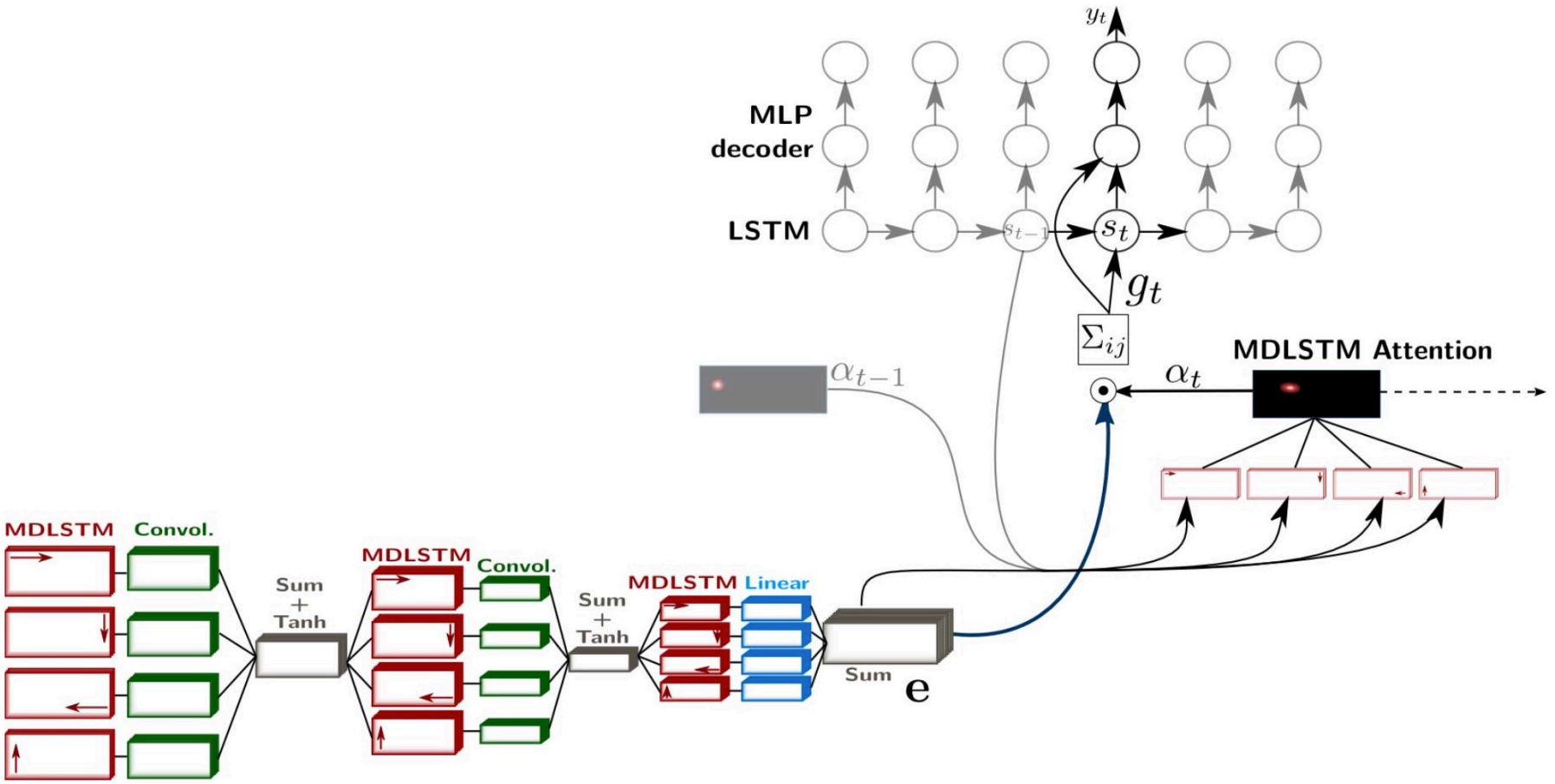


Image source: Bluche, T., Louradour, J., & Messina, R. (2017, November). Scan, attend and read: End-to-end handwritten paragraph recognition with MDLSTM attention. In 2017 14th IAPR international conference on document analysis and recognition (ICDAR) (Vol. 1, pp. 1050-1055). IEEE.

Scan, Attend & Read

The network is connected as follows:

$$F = \mathrm{MDLSTM}(x)$$

The encoder produces a 2D feature map F from the input image x.

$$\alpha_t = \mathrm{Attention}(F, \alpha_{t-1})$$

h_{t-1} is the decoder hidden state from the previous step.

At each decoding step t, the attention network uses the previous decoder hidden state h_{t-1} and previous attention map α_{t-1} to compute a new **spatial weight map** α_t over every cell of F.

$$s_t = \sum_i \alpha_{t,i} \, F_i$$

Scan, Attend & Read

The weighted sum **collapses the entire 2D feature map into a single context vector** s_t.

$$y_t,\ h_t = \mathrm{Decoder}(h_{t-1},\ s_t)$$

The decoder uses s_t together with its previous state to emit the next character y_t and update its hidden state h_t.

Scan, Attend & Read: Attention in a Single Line

Attention visualized for consecutive timesteps in a single text line:

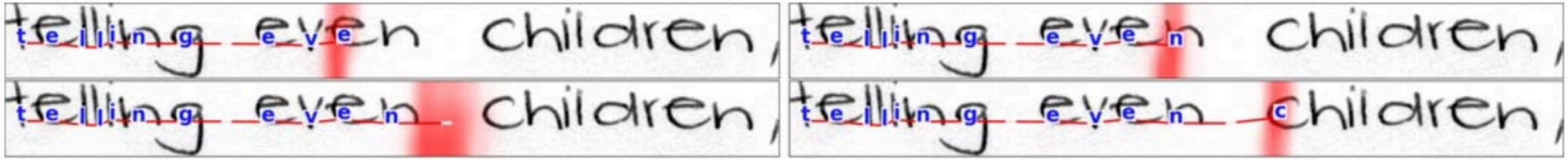


Image source: Bluche, T., Louradour, J., & Messina, R. (2017, November). Scan, attend and read: End-to-end handwritten paragraph recognition with MDLSTM attention. In 2017 14th IAPR international conference on document analysis and recognition (ICDAR) (Vol. 1, pp. 1050-1055). IEEE.

Reading Left to Right — Emergent Behaviour

The model is never explicitly instructed to read left-to-right or follow lines; it discovers this reading order from character sequence supervision.

Key observations in the single-line visualisation:

- Each heat map corresponds to **one decoding step** (one character emitted)
- The focus region moves **progressively from left to right** along the text line

Scan, Attend & Read: Attention in Multiple Lines

Attention visualized for consecutive timesteps in multiple text lines:

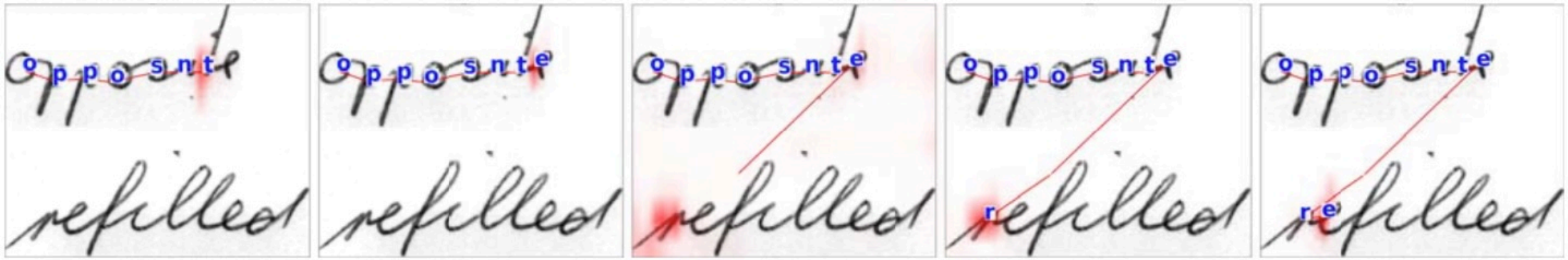


Image source: Bluche, T., Louradour, J., & Messina, R. (2017, November). Scan, attend and read: End-to-end handwritten paragraph recognition with MDLSTM attention. In 2017 14th IAPR international conference on document analysis and recognition (ICDAR) (Vol. 1, pp. 1050-1055). IEEE.

Line Transitions Without Explicit Segmentation

When the decoder reaches the end of one line it must jump to the beginning of the next — again **without any hard-coded rule**.

Why does this work?

- The MDLSTM feature map encodes **vertical position** implicitly: features near the top of the image differ systematically from features near the bottom
- After emitting all characters of a line, the attention state encodes "end of line reached"; the attention network then finds that the highest-scoring spatial region has **shifted to the next line**
- The model therefore learns a soft **line-break behaviour** from the sequence of (image, transcript) training pairs

Scan, Attend & Read: Attention in a Paragraph

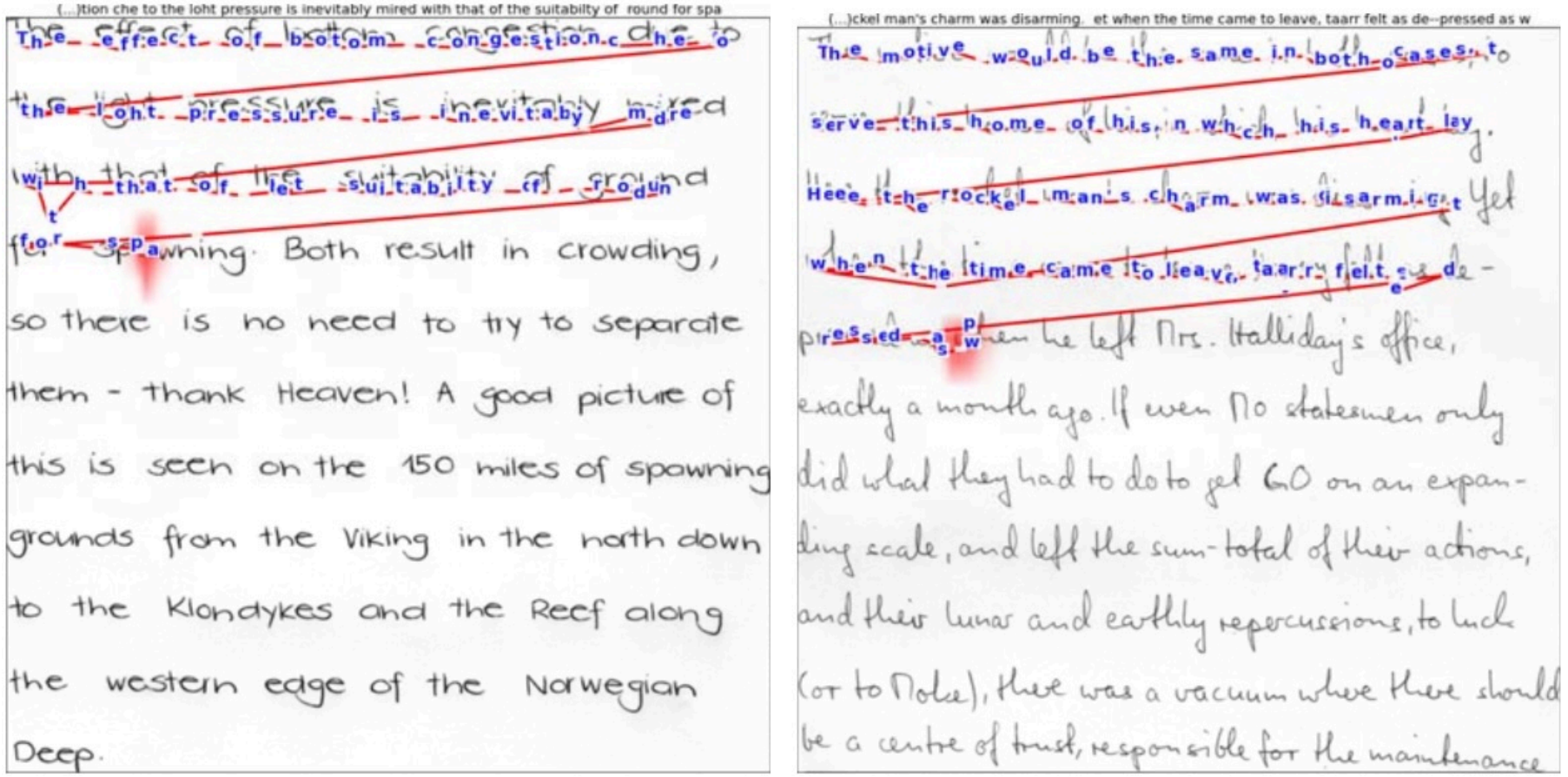


Image source: Bluche, T., Louradour, J., & Messina, R. (2017, November). Scan, attend and read: End-to-end handwritten paragraph recognition with MDLSTM attention. In 2017 14th IAPR international conference on document analysis and recognition (ICDAR) (Vol. 1, pp. 1050-1055). IEEE.

From Global Attention to Line Attention

Scan, Attend & Read shows that one global attention mechanism can learn to read complete paragraphs.

However, there is another way to tell the story: keep CTC for line transcription, but learn where the lines are instead of segmenting them by hand.

A Different Approach: Decompose the Problem

SAR uses a **single global attention** over the full 2D MDLSTM feature map.

An alternative is to **decompose** paragraph recognition into two sequential stages:

Stage	Question answered
1 — Line localisation	*Where* are the text lines in the image?
2 — Line transcription	*What* is written on each located line?

A Different Approach: Decompose the Problem

Bluche (2016) proposes to solve **both stages jointly and end-to-end** using attention:

- An **attention-based collapsing** layer compresses the 2D image into a sequence of 1D line feature vectors — implicitly finding line positions
- A **CTC transcription stage** then reads each collapsed line representation

Joint Line Segmentation and Transcription: Collapsing via Attention

Right side shows the attention network:

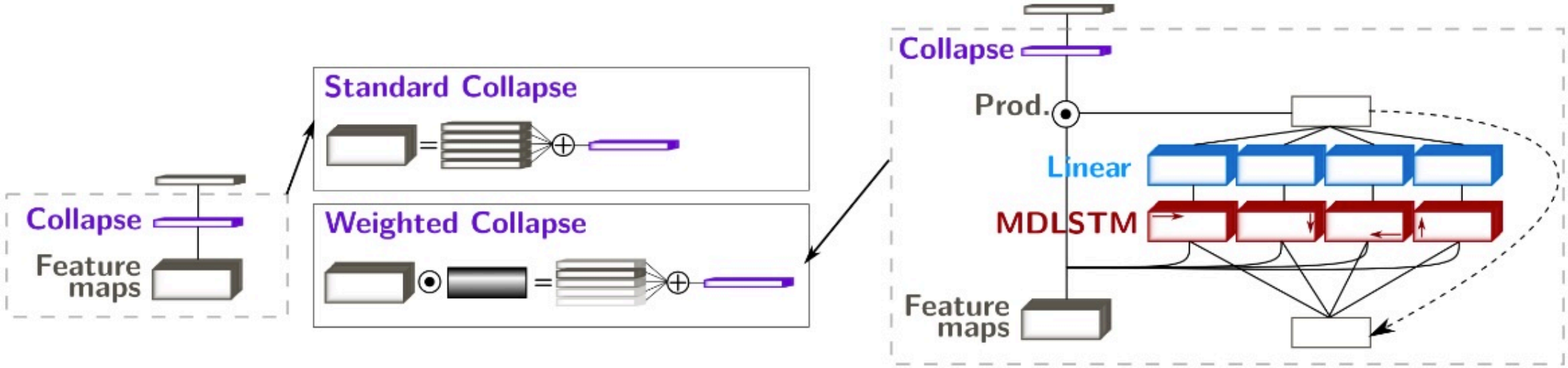


Image source: Bluche, T. (2016). Joint line segmentation and transcription for end-to-end handwritten paragraph recognition. Advances in neural information processing systems, 29.

Joint Line Segmentation and Transcription

The network is connected as follows:

$$F = \mathrm{MDLSTM}(x)$$

The encoder produces a 2D feature map $F \in \mathbb{R}^{W' \times H' \times d}$ from the input image x.

$$\beta_\ell = \mathrm{LineAttention}(F,\, \beta_{\ell-1})$$

An attention network uses the previous line-attention map $\beta_{\ell-1}$ together with the feature map F to produce new vertical attention weights β_ℓ over rows, indicating where line ℓ lies in the image.

$$\bar{F}_\ell = \sum_y \beta_{\ell,y}\, F_y$$

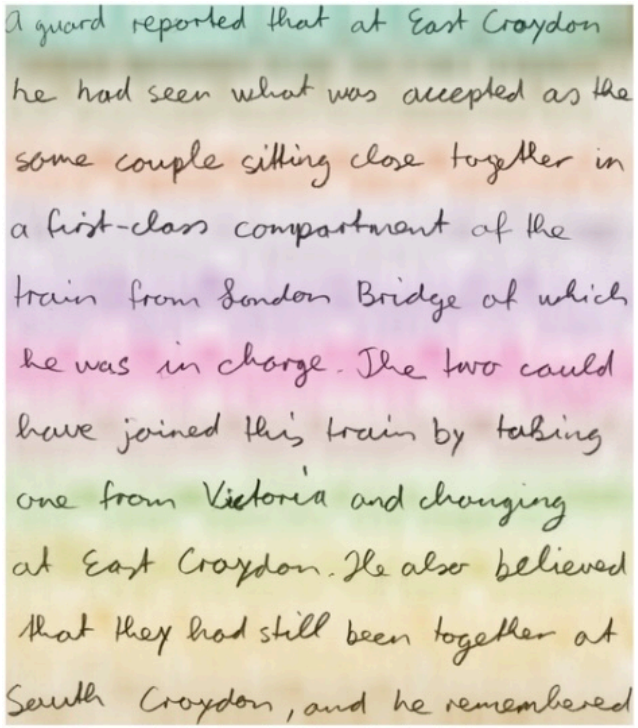
Joint Line Segmentation and Transcription

The weighted sum **collapses the 2D feature map into a 1D horizontal strip** $\bar{F}_\ell \in \mathbb{R}^{W' \times d}$ — one strip per text line. Unlike SAR, the result is still a sequence (along the width dimension), not a single vector.

$$p_\ell = \mathrm{CTC}(\bar{F}_\ell)$$

A CTC decoder reads left-to-right across the collapsed strip $\bar{F}_\ell$ and produces the character sequence for line ℓ.

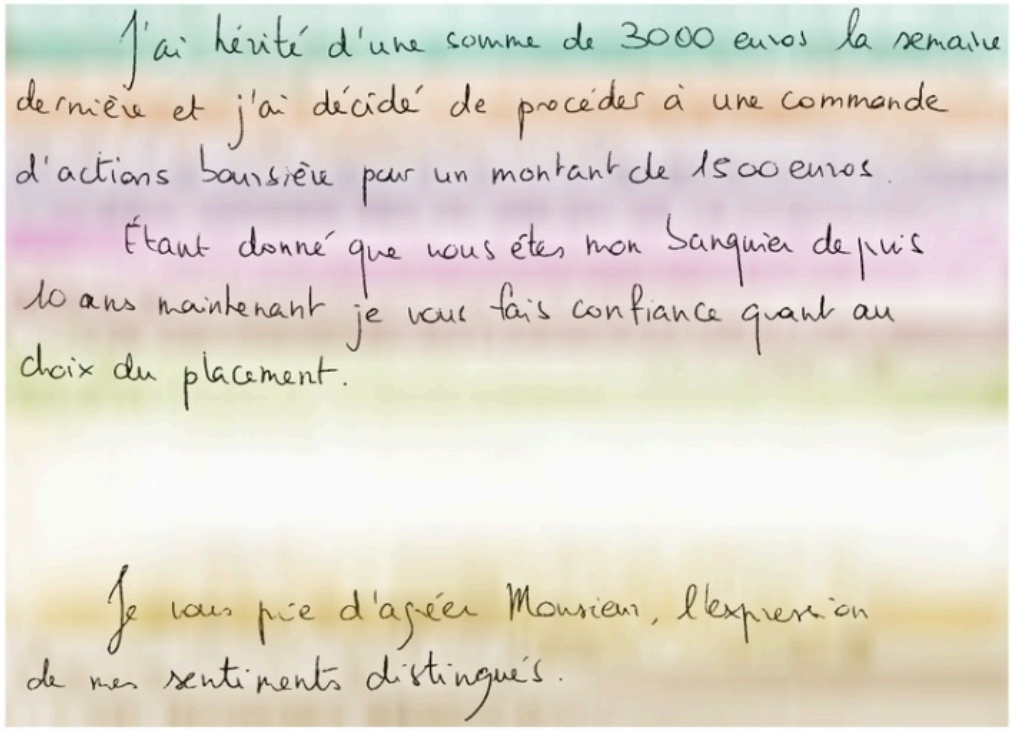
Joint Line Segmentation and Transcription: English Example



A guard reported that at East Craydon he had seen what was ouepted as the some couple sitting close togther in a first-class compartment of the train from London Bridge of which he was in charge . The two could have joined this train by taking one from Vectorin and changing at East Craydon . He also believed that they had still been together at South Croydon , and he remembered

Image source: Bluche, T. (2016). Joint line segmentation and transcription for end-to-end handwritten paragraph recognition. Advances in neural information processing systems, 29.

Joint Line Segmentation and Transcription: French Example



J'ai hérité d'une somme de 3000 euros la semaine dernière et j'ai décidé de procéder à une commande d'actions boursière pour un montant de 1500 euros. Etant donné que vous êtes mon banquier depuis 10 ans maintenant je vous fais confiance quant au choix du placement. Je vous prie d'agréer Monsieur, l'expression de mes sentiments distingués.

Image source: Bluche, T. (2016). Joint line segmentation and transcription for end-to-end handwritten paragraph recognition. Advances in neural information processing systems, 29.

What We Have Learned: Attention for Handwriting

Both papers demonstrate the same core insight — attention enables paragraph recognition — but approach it from different architectural angles.

	Scan, Attend & Read (2017)	Joint Line Seg. & Transcription (2016)
Encoder	MDLSTM (4-directional 2D LSTM)	MDLSTM-RNN
Attention	2D soft attention over full feature map	Vertical soft attention per line
Decoder	Character RNN decoder	CTC per collapsed line
Reading order	Learned end-to-end	Learned end-to-end
Line segmentation	Implicit (attention moves between lines)	Explicit via collapse weights

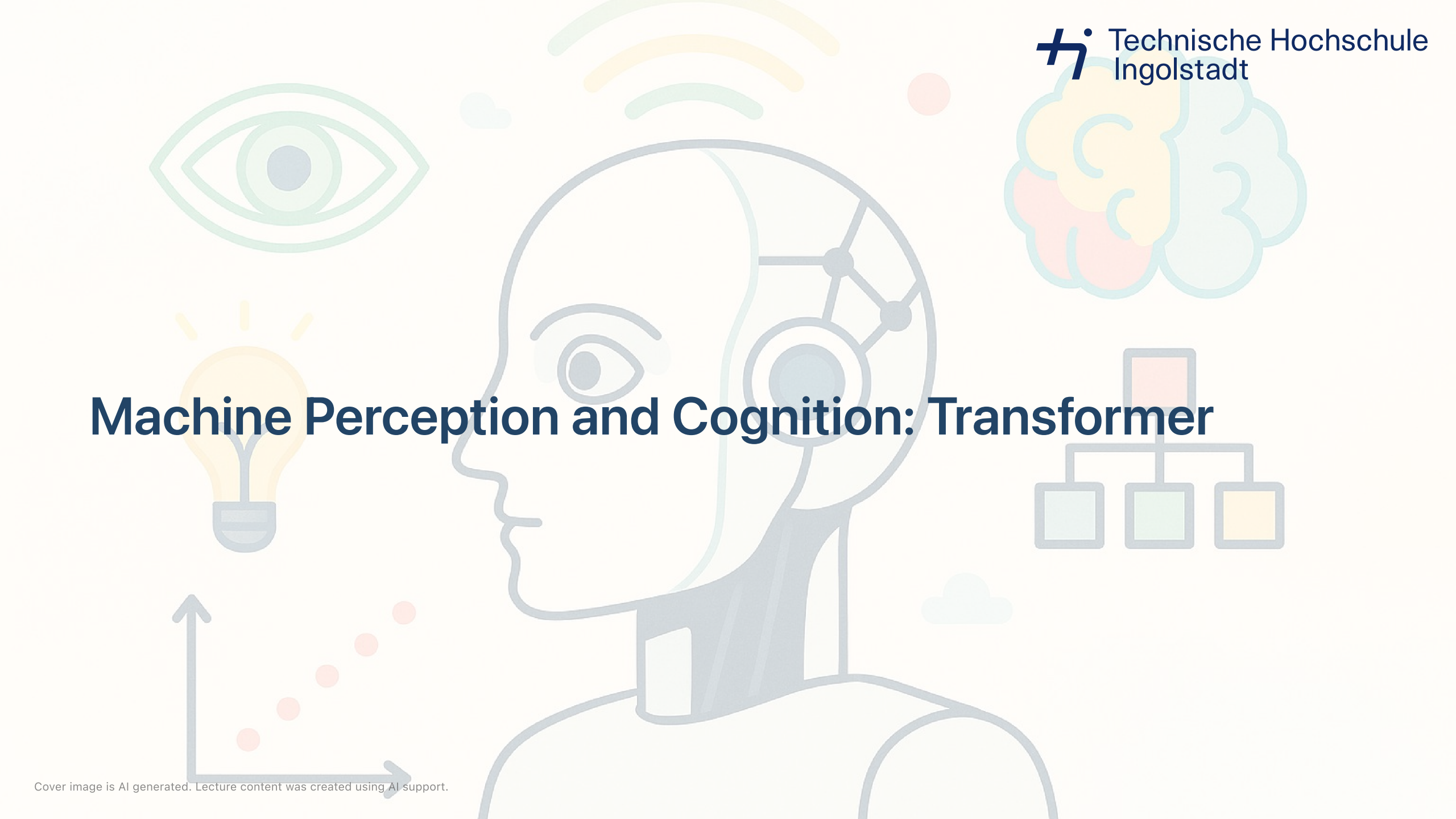
What We Have Learned: Attention for Handwriting

Key takeaways:

- Attention replaces hand-crafted segmentation with a **learnable, differentiable** alignment
- The model discovers reading order, line boundaries, and character extent **purely from (image, text) pairs**
- This closes the gap CTC left open: paragraph-level recognition becomes a single end-to-end trainable system

Technische Hochschule Ingolstadt

Machine Perception and Cognition: Transformer



Cover image is AI generated. Lecture content was created using AI support.

From RNNs to Transformers

Recurrent neural networks process sequences **step by step** — each hidden state depends on the previous one:

- **Sequential bottleneck**: processing of step t must wait for step $t-1$ — no parallelism during training
- **Vanishing gradients**: long-range dependencies are hard to capture even with LSTM
- **Fixed context**: the hidden state has limited capacity to carry information over very long sequences

From RNNs to Transformers

Transformer (Vaswani et al., 2017) addresses all three limitations:

- Processes the **entire sequence in parallel** using self-attention
- Connects **any two positions** directly — no intermediate steps
- Scales to very long sequences and very large models

Key Ideas of the Transformer

The Transformer is built on three fundamental ideas:

1. **Token embeddings** — represent discrete symbols as dense continuous vectors
2. **Positional encoding** — inject order information, since attention without positions is permutation-equivariant
3. **Multi-head self-attention** — each position directly attends to all others in parallel

These ideas are combined in stacked **encoder** and **decoder** blocks, each with residual connections and layer normalization, forming a flexible and highly scalable architecture.

What is Tokenization?

Before a neural network can process text, raw characters must be converted to discrete **tokens**:

$$\text{raw text} \xrightarrow{\text{tokenizer}} \text{sequence of token IDs} \xrightarrow{\text{embedding}} \text{sequence of vectors}$$

A **token** is the basic unit of text the model sees. It can be:

- A full word: `"transformer"` → 1 token
- A subword unit: `"transformer"` → `["transform", "er"]` → 2 tokens
- A single character

Most modern large language models use **subword tokenization**, which balances vocabulary size with coverage of rare and unseen words.

Word-Level vs. Subword Tokenization

Property	Word-level	Subword (BPE)
Vocabulary size	Very large (millions)	Controlled (30k–100k)
Unknown words	Frequent	Rare
Morphology	Ignored	Partially captured
Efficiency	Low for rare words	High

Problem with word-level: `"unhappy"`, `"unhappiness"`, `"unhappily"` are three unrelated tokens — the shared stem `"happy"` and prefix `"un"` are lost.

Subword tokenization decomposes words into reusable pieces: `["un", "happy", "ness"]` — shared morphemes are represented by shared tokens.

Byte Pair Encoding (BPE)

Byte Pair Encoding (Sennrich et al., 2016) is the most widely used subword tokenization algorithm:

Algorithm:

1. Start with a character-level vocabulary
2. Count all adjacent symbol pairs in the training corpus
3. Merge the most frequent pair into a new symbol
4. Repeat until the vocabulary reaches the target size

Typical subword vocabularies used in practice range from **30,000 to 100,000 tokens**, balancing coverage with model size.

The Vocabulary

The tokenizer produces a **vocabulary** $\mathcal{V}$ — a finite set of tokens with size $|\mathcal{V}|$.

Each token is assigned an integer ID:

$$\text{"the"} \mapsto 464, \quad \text{"cat"} \mapsto 3797, \quad \text{"sat"} \mapsto 3332$$

Special tokens are also included: `[BOS]` (beginning of sequence), `[EOS]` (end of sequence), `[PAD]` (padding), `[UNK]` (unknown).

A sentence is encoded as a sequence of IDs, forming the input to the embedding layer:

$$\text{"The cat sat"} \mapsto [464,\ 3797,\ 3332]$$

Token Embeddings

Token IDs are discrete integers. A **token embedding** maps each ID to a dense real-valued vector:

$$\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$$

The embedding matrix $\mathbf{E}$ has one row per vocabulary token. For a token with ID k, the embedding is the k-th row:

$$\mathbf{x}_k = \mathbf{E}[k, :] \in \mathbb{R}^d$$

This is a **linear** lookup — during training, the rows of $\mathbf{E}$ are updated by gradient descent so that semantically similar tokens have similar embedding vectors.

The Embedding Matrix

The embedding dimension d is a key hyperparameter — typical values range from $d = 512$ for small research models up to $d = 12,288$ for very large models.

For an input sequence of n tokens, the embedding layer produces a matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$:

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_n^\top \end{pmatrix}$$

The Ordering Problem

Attention without positional information is **permutation-equivariant**: if the input tokens are reordered, the outputs are reordered in the same way.

For a permutation matrix $\mathbf{P}$:

$$\text{Attention}(\mathbf{PX}) = \mathbf{P}\,\text{Attention}(\mathbf{X})$$

Without additional information, the Transformer cannot tell whether token i appears before or after token j.

Solution: add a **positional encoding** vector to each token embedding before feeding the sequence into the attention layers. This injects order information into the representations.

Sinusoidal Positional Encoding

The original Transformer (Vaswani et al., 2017) uses **deterministic sinusoidal encoding**. For position $pos \in \{0, 1, \ldots, n-1\}$ and embedding dimension index $i \in \{0, 1, \ldots, d/2 - 1\}$:

$$PE(pos,\, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos,\, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

- The denominator $10000^{2i/d}$ spans a wide range of frequencies across the d dimensions
- **No parameters are learned** — the encoding is fixed and added element-wise to the token embeddings: $\mathbf{z}_i = \mathbf{x}_i + PE(i, :)$

Sinusoidal Positional Encoding

The same sinusoidal formula is used in diffusion models to encode the **noise timestep** t (as covered in the diffusion models chapter):

$$\text{emb}(t)_{2i} = \sin\left(\frac{t}{10000^{2i/d}}\right), \quad \text{emb}(t)_{2i+1} = \cos\left(\frac{t}{10000^{2i/d}}\right)$$

In both cases, the 10000 is a fixed scale hyperparameter that spreads sinusoidal frequencies across the embedding dimensions — lower dimensions encode fast-changing (local) patterns, higher dimensions encode slow-changing (global) patterns.

Positional Encoding

Absolute positional sinusoidal encoding:

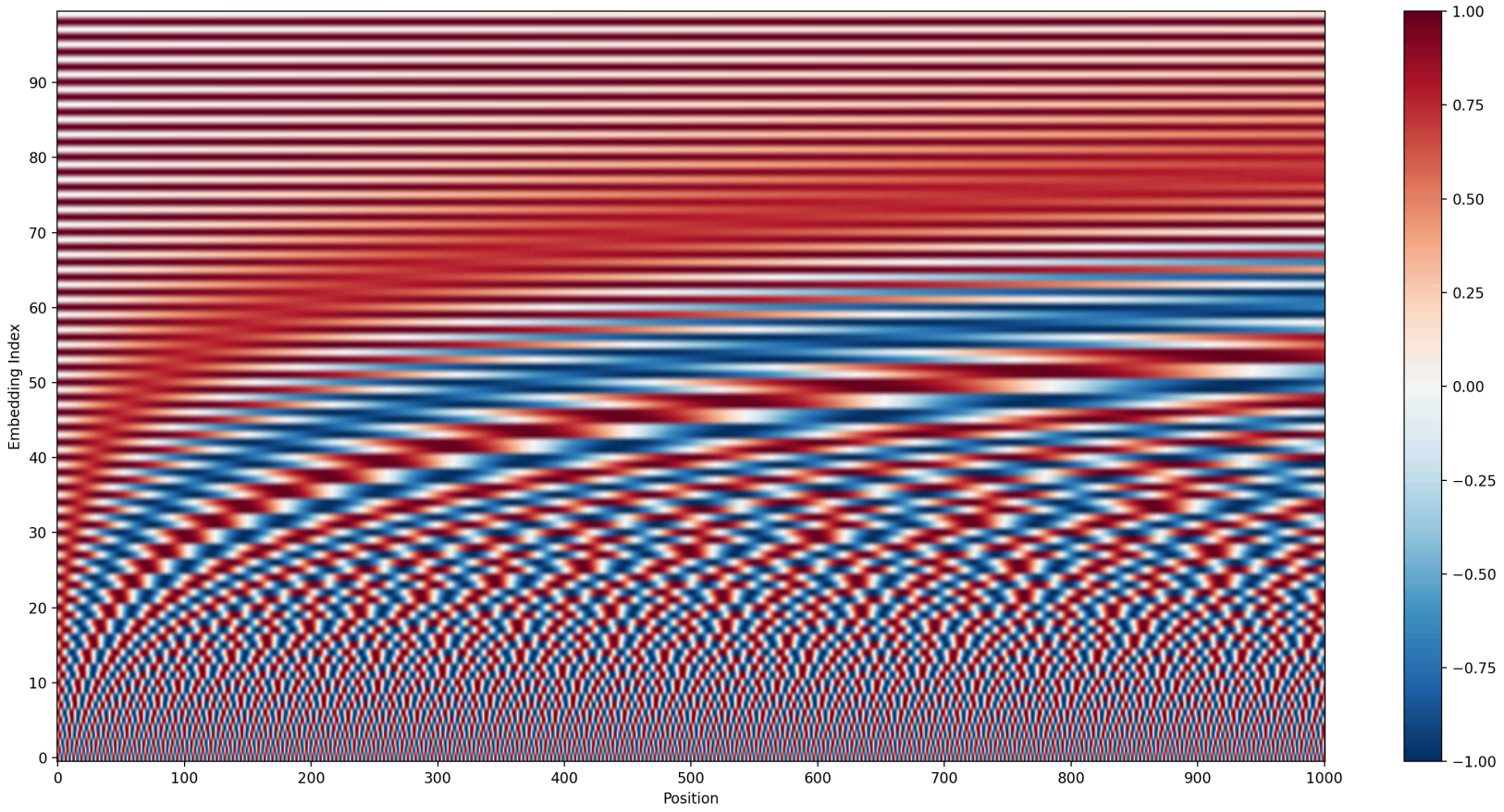


Image source: Nils Blümer, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Absolute_positional_encoding.png

Sinusoidal Encoding Properties

Key properties of sinusoidal positional encoding:

- **Unique per position**: each position pos receives a distinct encoding vector in $\mathbb{R}^d$
- **Bounded**: all values are in $[-1, 1]$ — no scaling issues regardless of sequence length
- **Generalizes beyond training length**: the formula applies to any pos, even positions not seen during training

One Encoder Layer

An encoder layer with self-attention:

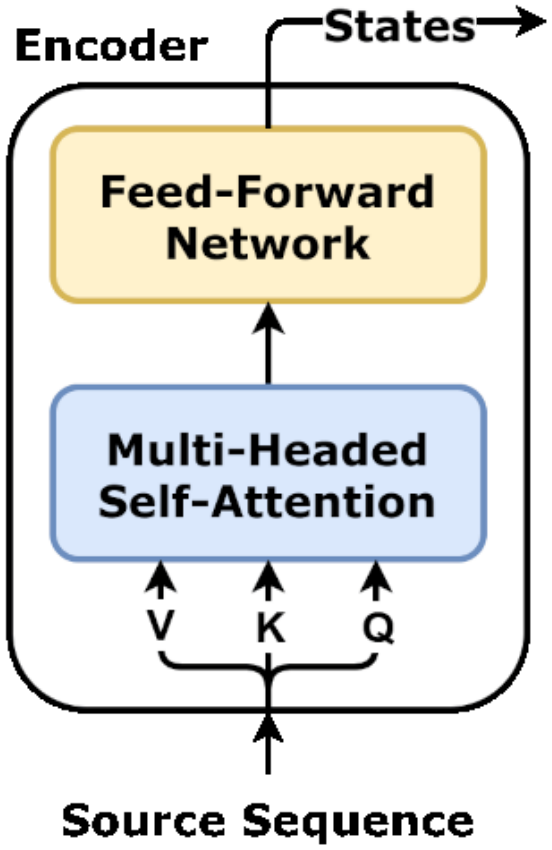


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_one_encoder_block.png

One Decoder Layer

A decoder layer with masked self-attention and cross-attention:

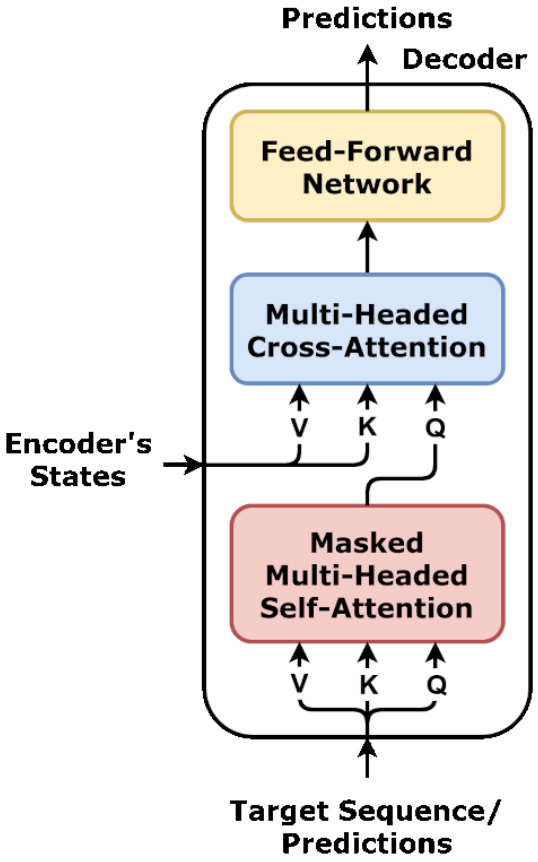


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_one_decoder_block.png

Encoder-Decoder Block

A single encoder-decoder block with self, cross and masked multi-head attention:

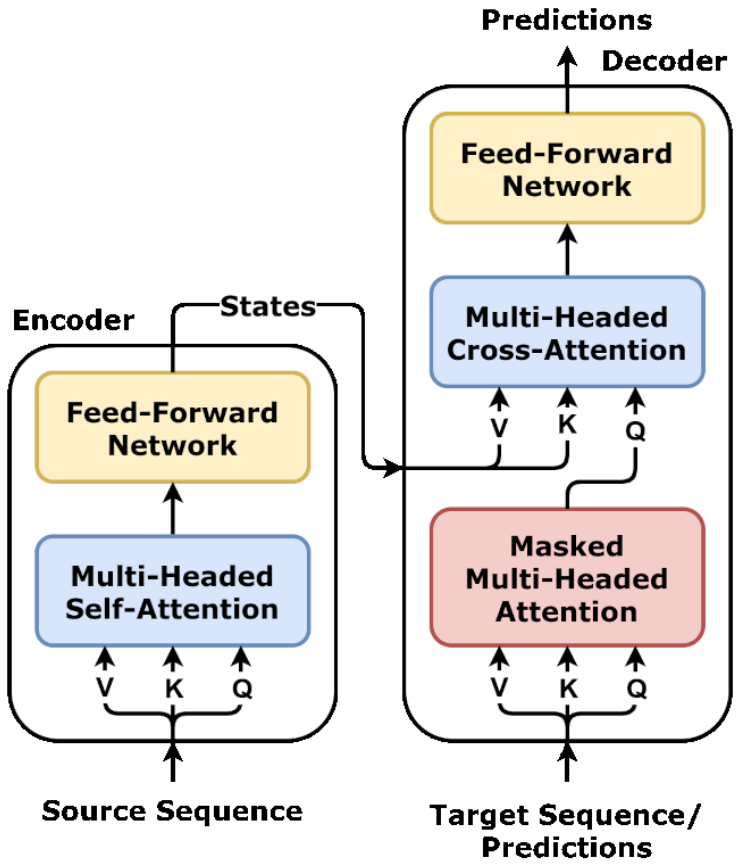


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_one_encoder-decoder_block.png

Attention Revisited

In the attention chapter, we introduced **scaled dot-product attention** (Vaswani et al., 2017) with query matrix $\mathbf{Q}$, key matrix $\mathbf{K}$ and value matrix $\mathbf{V}$:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

- $\mathbf{Q} \in \mathbb{R}^{n \times d_k}$, $\mathbf{K} \in \mathbb{R}^{m \times d_k}$, $\mathbf{V} \in \mathbb{R}^{m \times d_v}$, where n and m are the number of tokens in the query and key/value sequences, respectively — **each row corresponds to one token**
- The product $\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{n \times m}$ is a token-to-token score matrix: entry (i, j) measures how strongly token i attends to token j along the sequence
- **Self-attention**: $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ all originate from the same input sequence via learned projection matrices $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V$

Self-Attention in the Encoder

In the encoder, every position attends to **all other positions** in the input — no masking.

Given positioned input $\mathbf{Z} \in \mathbb{R}^{n \times d}$ (token embeddings with positional encodings added), where each of the n rows of $\mathbf{Z}$ corresponds to one token in the sequence:

$$\mathbf{Q} = \mathbf{Z}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{Z}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{Z}\mathbf{W}^V$$

$$\mathbf{O} = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

Since every position has full access to every other position, encoder self-attention is **bidirectional** — making the encoder well-suited for building rich contextual representations of the full input sequence.

Masked Self-Attention in the Decoder

During autoregressive generation the decoder must not attend to future tokens. A **causal mask** is applied before softmax (as introduced in the attention chapter):

$$M_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

$$\mathbf{O} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}$$

After softmax, the $-\infty$ entries become exactly 0, so position i only aggregates information from positions $1, \ldots, i$. This **unidirectional** (causal) attention supports both parallel training via teacher forcing and sequential autoregressive inference.

Attention in a Transformer

A single attention head with scaled dot product attention scores:

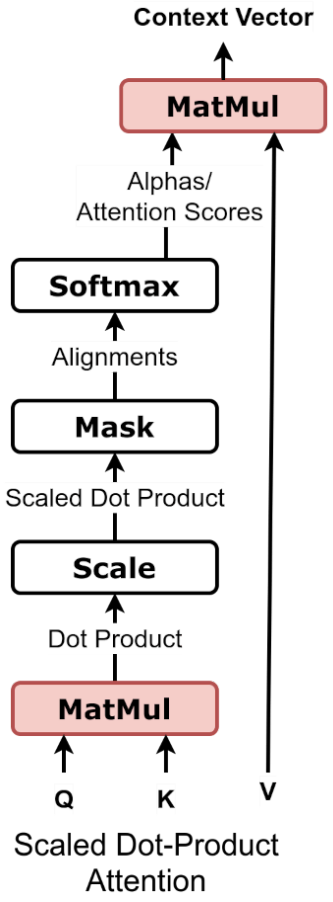


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_attention_block_diagram.png

Multi-Head Attention in a Transformer

Multi-head attention by concatenating independent heads:

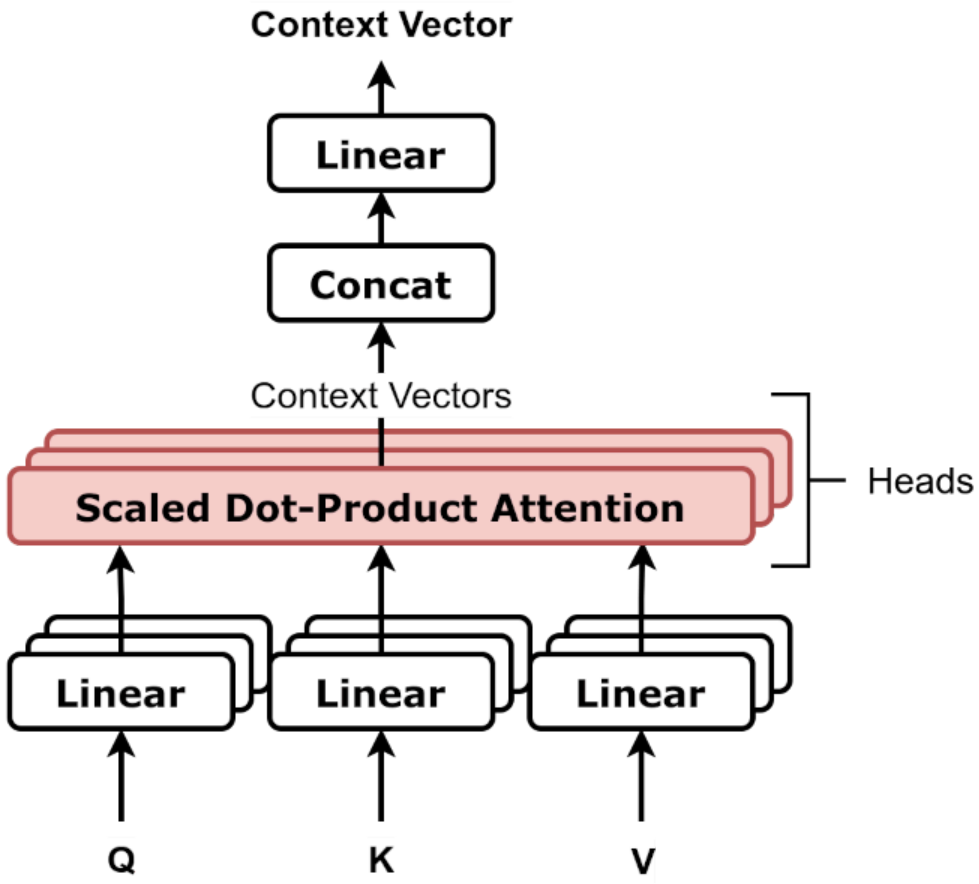


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Multiheaded_attention,_block_diagram.png

Cross-Attention in the Transformer

The decoder uses **cross-attention** to read from the encoder's output. Queries come from the decoder, keys and values come from the encoder:

$$\mathbf{Q} = \mathbf{H}^{(\text{dec})}\mathbf{W}^{Q}, \quad \mathbf{K} = \mathbf{H}^{(\text{enc})}\mathbf{W}^{K}, \quad \mathbf{V} = \mathbf{H}^{(\text{enc})}\mathbf{W}^{V}$$

$$\mathbf{O} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_k}}\right)\mathbf{V}$$

- $\mathbf{H}^{(\text{dec})}$: decoder hidden states after masked self-attention
- $\mathbf{H}^{(\text{enc})}$: encoder hidden states from the top encoder layer

Each decoder position attends to **all encoder positions**, learning which parts of the source are most relevant for predicting the current target token.

Cross-Attention: Q, K, V Sources

Component	Source
Queries $\mathbf{Q}$	Decoder hidden states
Keys $\mathbf{K}$	Encoder hidden states
Values $\mathbf{V}$	Encoder hidden states

This is the same cross-attention mechanism introduced in the attention chapter — the key–value source is the encoder, while the query source is the decoder.

This allows the decoder to **condition on the full encoded source sequence** at every decoding step.

Encoder-Decoder Block

A single encoder-decoder block with self, cross and masked multi-head attention:

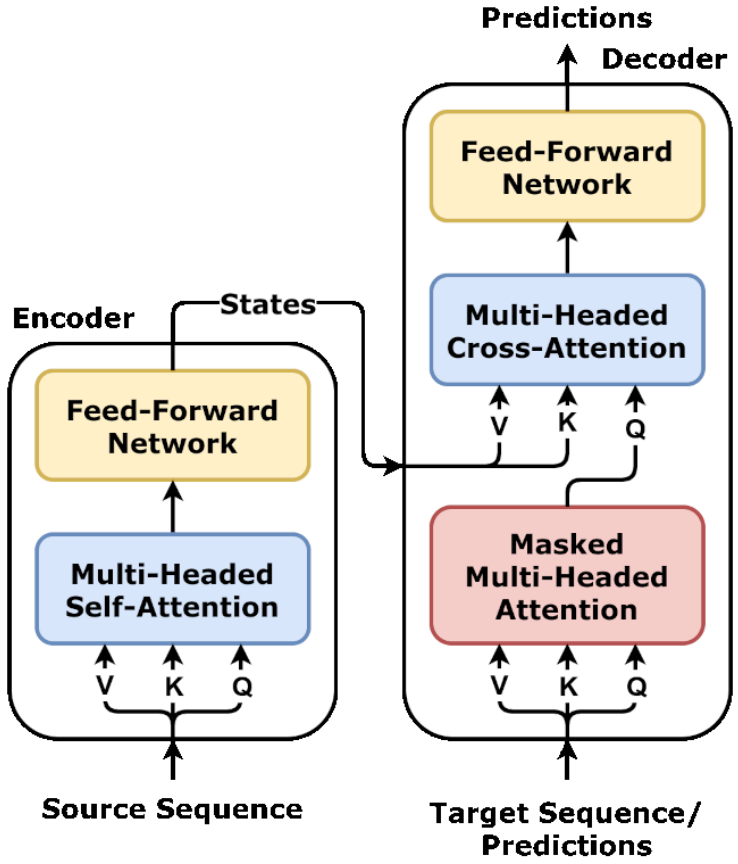


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_one_encoder-decoder_block.png

Feed-Forward Sublayer

Each encoder and decoder layer contains a **position-wise feed-forward network** (FFN), applied **independently** to each sequence position:

$$\mathrm{FFN}(\mathbf{z}) = \mathbf{W}_2\,\sigma(\mathbf{W}_1\mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2$$

where σ is a non-linear activation function (ReLU in the original paper).

- $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{ff}} \times d}$, $\mathbf{W}_2 \in \mathbb{R}^{d \times d_{\text{ff}}}$ — the inner dimension is typically $d_{\text{ff}} = 4d$
- The FFN introduces **non-linearity** and expands the model's representational capacity
- The same weight matrices are shared across all positions in the sequence

Stacking Encoder and Decoder Layers

A single encoder or decoder layer provides limited representational depth. In the full Transformer, N layers are **stacked**:

- **Encoder stack**: N identical encoder layers applied in sequence; each layer refines the representations with self-attention and FFN
- **Decoder stack**: N identical decoder layers; each applies masked self-attention, cross-attention over the encoder output, and FFN

Stacked Encoder-Decoder Transformers

A single-layer and stacked encoder-decoder transformer side-by-side:

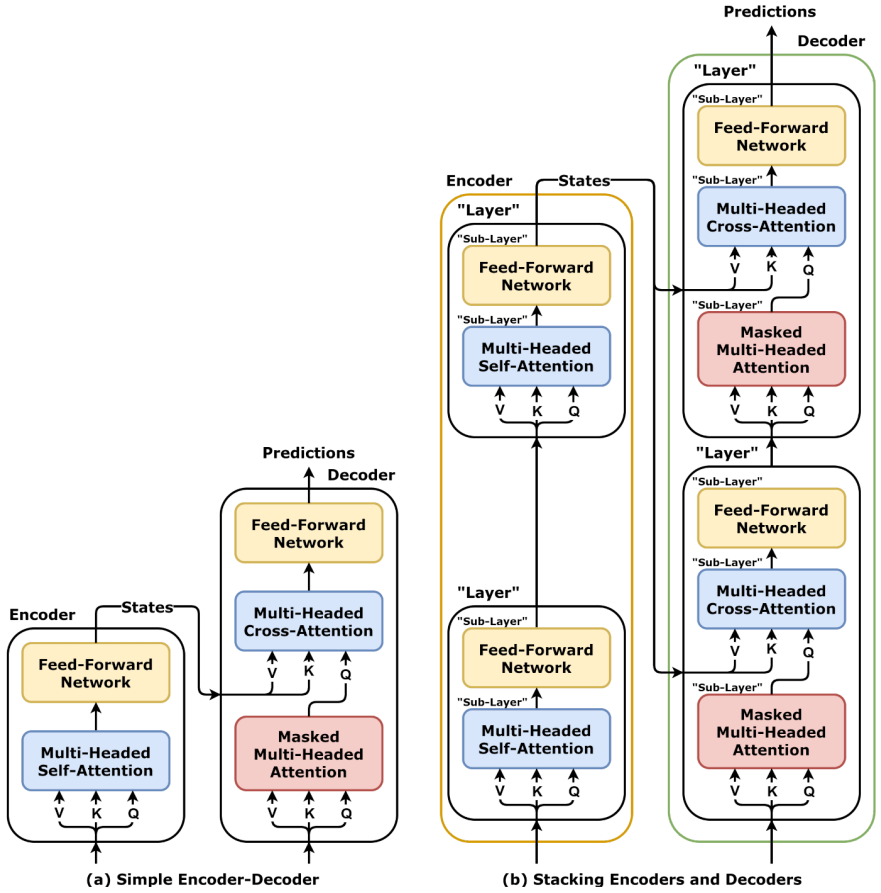


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_stacked_multilayers.png

Transformer Networks

A transformer with multi-layered encoder-decoder networks:

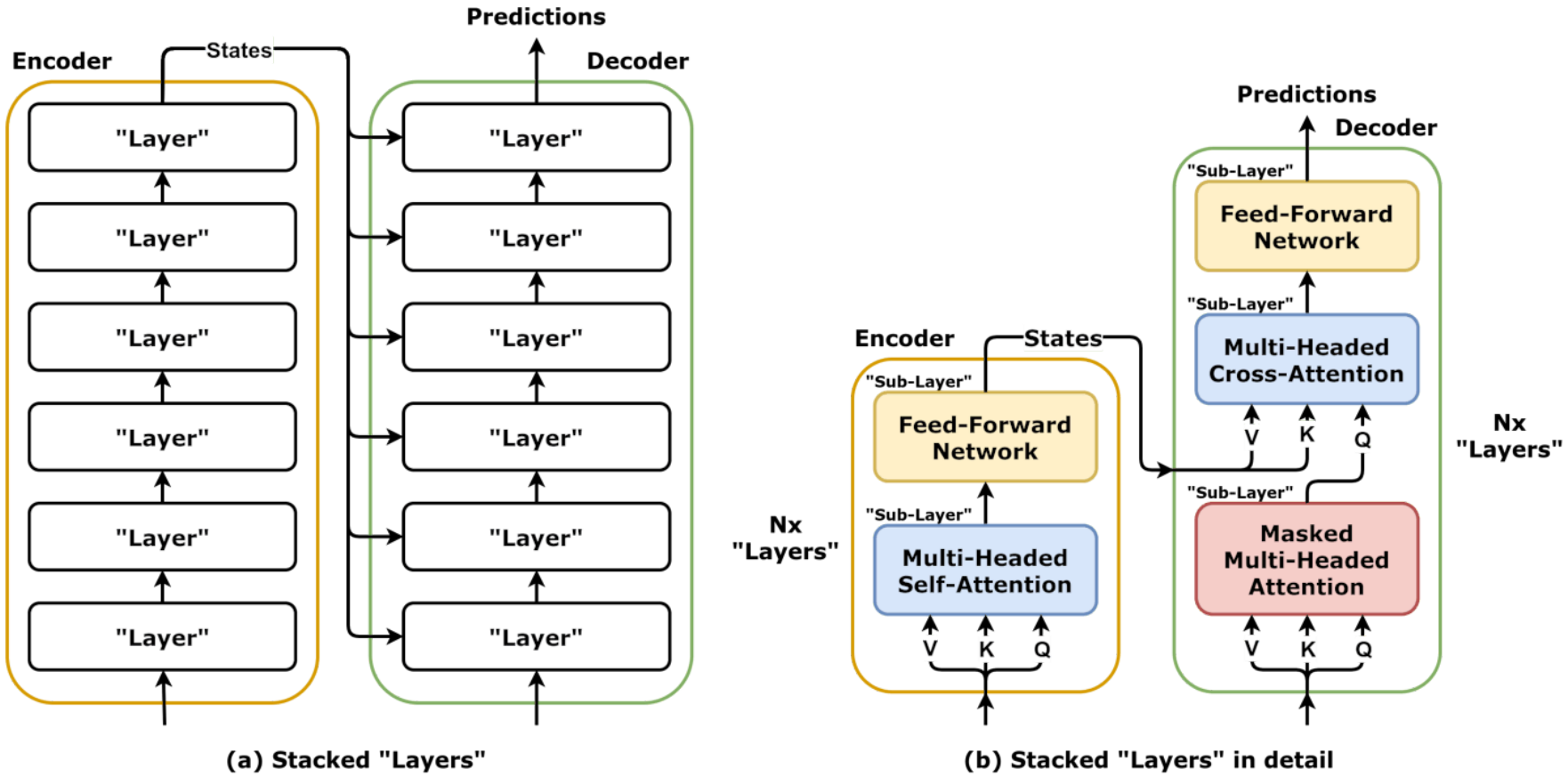


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_stacked_layers_and_sublayers.png

Transformer Networks

A standard transformer with embedding, positional encoding, encoder-decoder blocks and linear prediction:

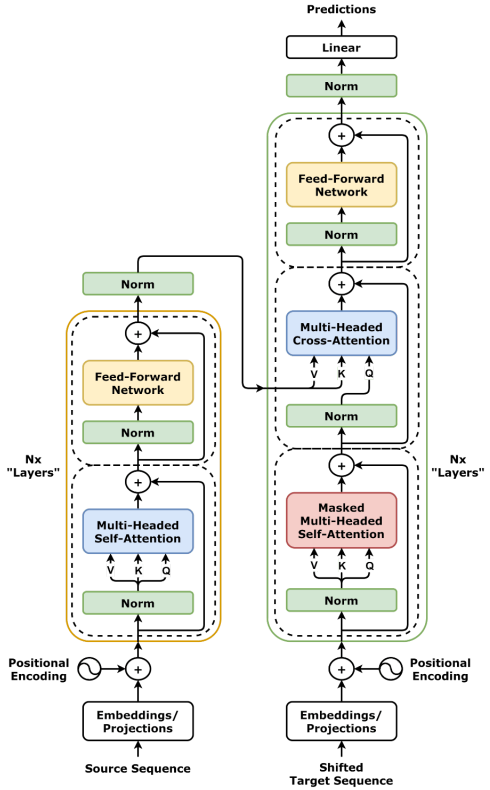


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_full_architecture.png

Encoder-Only Transformers

Encoder-only transformers use only the encoder stack. All positions attend to all others (bidirectional self-attention):

- Pre-trained with **masked language modeling** — randomly mask a fraction of tokens and predict them from the full context
- Well-suited for **understanding** tasks: classification, named entity recognition
- **Not suitable** for autoregressive generation — the model sees the full sequence at once

Applications: sentiment analysis, semantic search, text classification.

Decoder-Only Transformers

Decoder-only transformers use only the decoder stack with masked (causal) self-attention:

- Pre-trained with **next-token prediction** — predict token $t+1$ given tokens $1, \ldots, t$
- All positions attend only to previous positions — **unidirectional**
- Well-suited for **text generation**: language modeling, code completion, dialogue

At scale, decoder-only models exhibit **emergent capabilities**: few-shot learning, chain-of-thought reasoning, and in-context learning without weight updates.

Most modern large language models are decoder-only architectures.

GPT is Decoder-Only

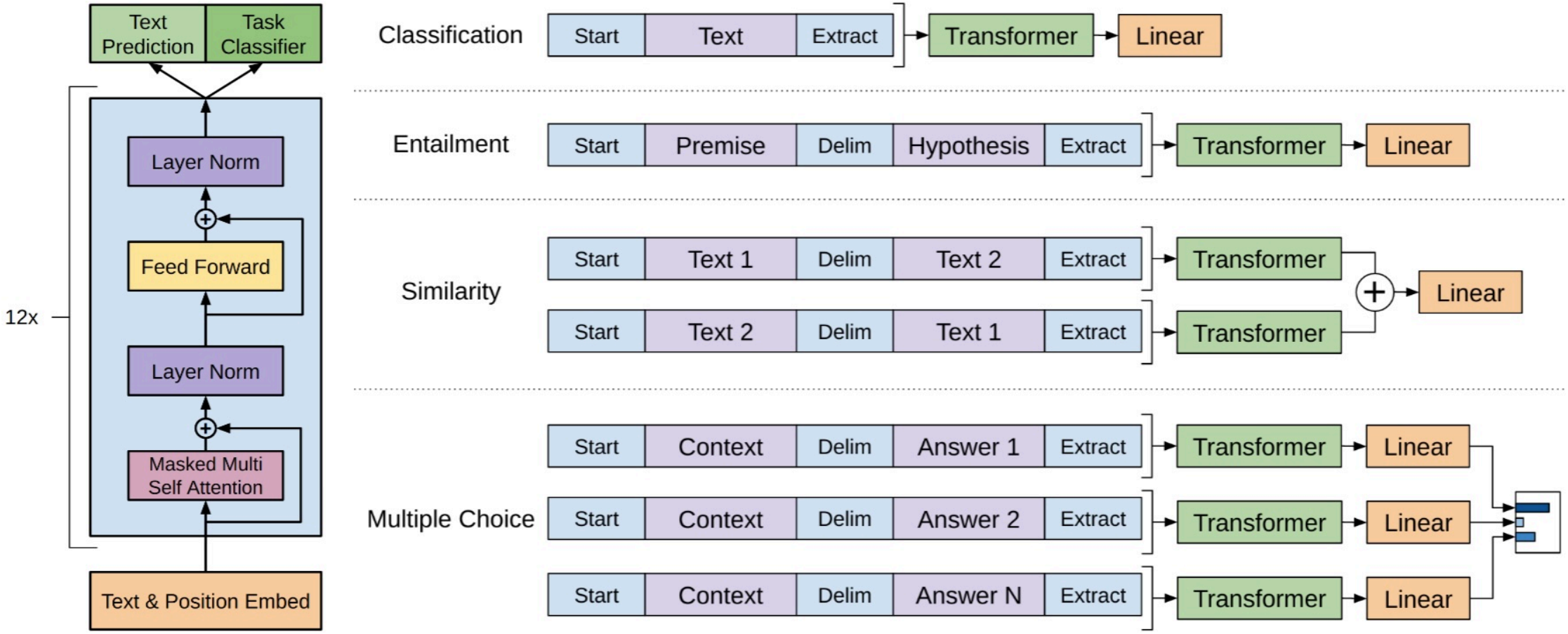


Image source: Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving language understanding by generative pre-training.

Encoder-Decoder (Seq2Seq) Transformers

Encoder-decoder transformers use both stacks — the architecture introduced by Vaswani et al. (2017):

Well-suited for **sequence-to-sequence** tasks: machine translation, summarization, question generation.

The encoder processes the full source with bidirectional attention; the decoder generates the target autoregressively, attending to the encoder output via cross-attention at every step.

Autoregressive Language Modeling

The model predicts the next token distribution from the current hidden state $\mathbf{h}_{i-1}$ using an output projection:

$$\hat{P}(t_i \mid t_1, \ldots, t_{i-1}) = \mathrm{softmax}(\mathbf{W}_{\text{out}}\, \mathbf{h}_{i-1})$$

where $\mathbf{W}_{\text{out}} \in \mathbb{R}^{|\mathcal{V}| \times d}$

Training a Transformer

In language models, the target sequence is the input sequence shifted by one token:

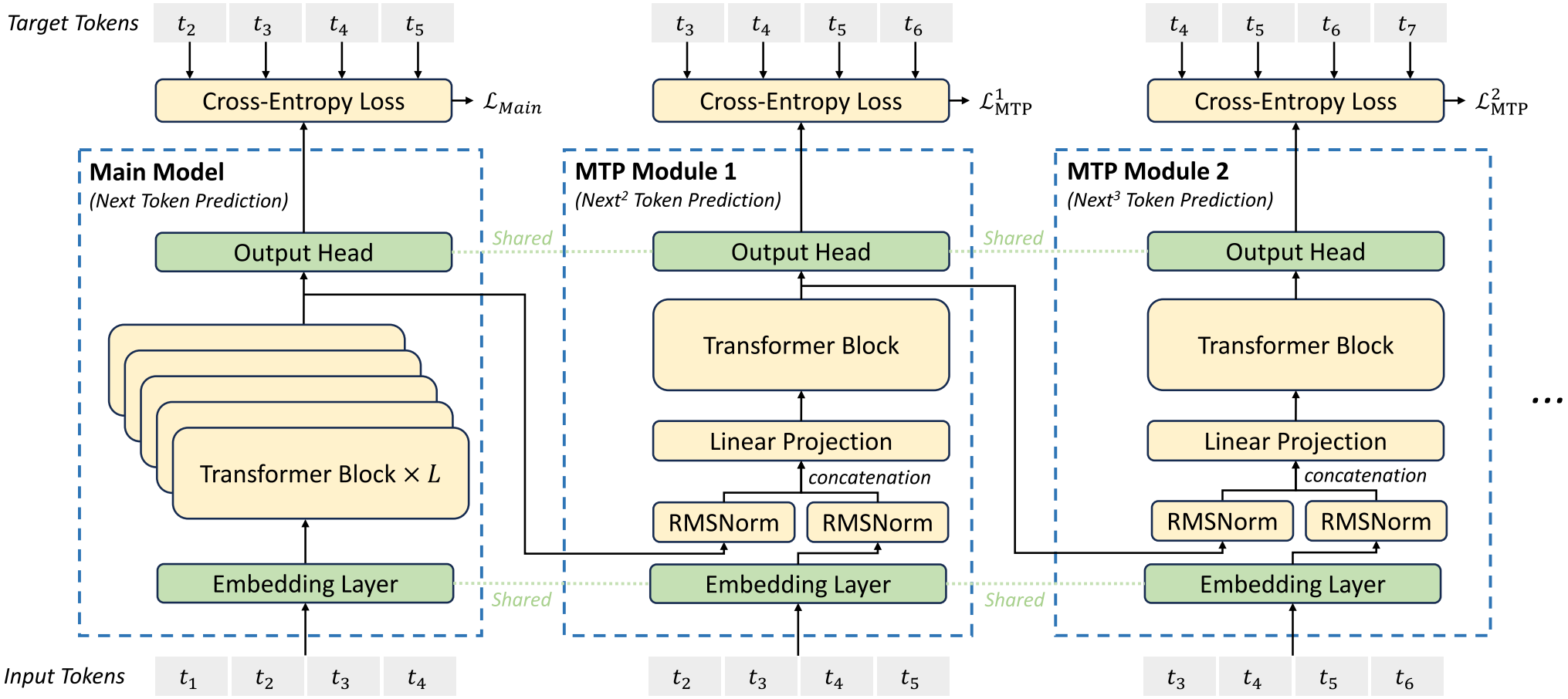


Image source: DeepSeek, MIT , via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Multi-Token_Prediction_(DeepSeek)_01.svg

Teacher Forcing

During training, the decoder receives the **ground-truth** tokens as input at each step — not its own previous predictions. This is called **teacher forcing**:

- At training step i, the decoder input is the ground-truth token t_{i-1}
- The decoder predicts a probability distribution over $\mathcal{V}$ for position i
- All positions are trained **in parallel** using the causal mask — enabling efficient GPU utilization

Teacher Forcing

At **inference** time, the model generates autoregressively:

1. Feed the start token `[BOS]`
2. Predict $\hat{t}_2 = \arg\max \hat{P}(\cdot \mid t_1)$ (or sample stochastically)
3. Append $\hat{t}_2$ and predict $\hat{t}_3$
4. Repeat until `[EOS]` or maximum length

Teacher forcing enables efficient parallel training while maintaining the autoregressive structure at inference.

Training Loss: Cross-Entropy

The training objective is to maximize the log-likelihood of the correct next token at every position. This is equivalent to minimizing the **cross-entropy loss**:

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n} \log \hat{P}(t_i \mid t_1, \ldots, t_{i-1})$$

For a vocabulary of size $|\mathcal{V}|$, this can be written as a standard cross-entropy between the predicted distribution $\hat{\mathbf{y}}_i \in \mathbb{R}^{|\mathcal{V}|}$ and the one-hot target $\mathbf{y}_i$:

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n} \mathbf{y}_i^\top \log \hat{\mathbf{y}}_i$$

LLaMa2 Training

Training loop of pretraining, finetuning and reinforcement learning in LLaMa2:

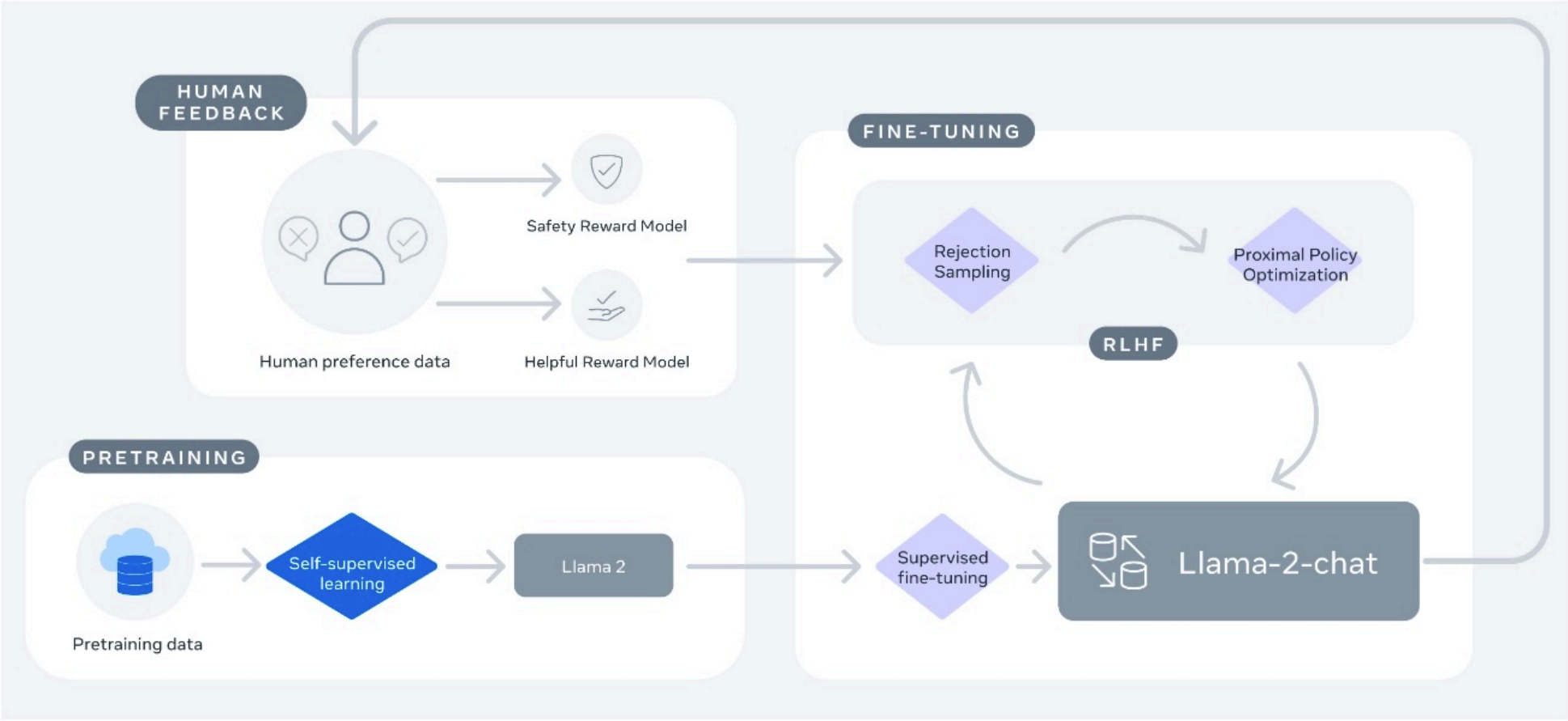


Image source: Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., ... & Scialom, T. (2023). Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288.

Training at Scale

Large language models are trained on massive corpora with billions to trillions of tokens. Model sizes range from hundreds of millions to hundreds of billions of parameters.

Scaling laws (Kaplan et al., 2020) show that loss decreases as a predictable power law in both model size and dataset size — motivating ever-larger training runs.

Large-batch distributed training across hundreds or thousands of GPUs is standard.

Training at Scale

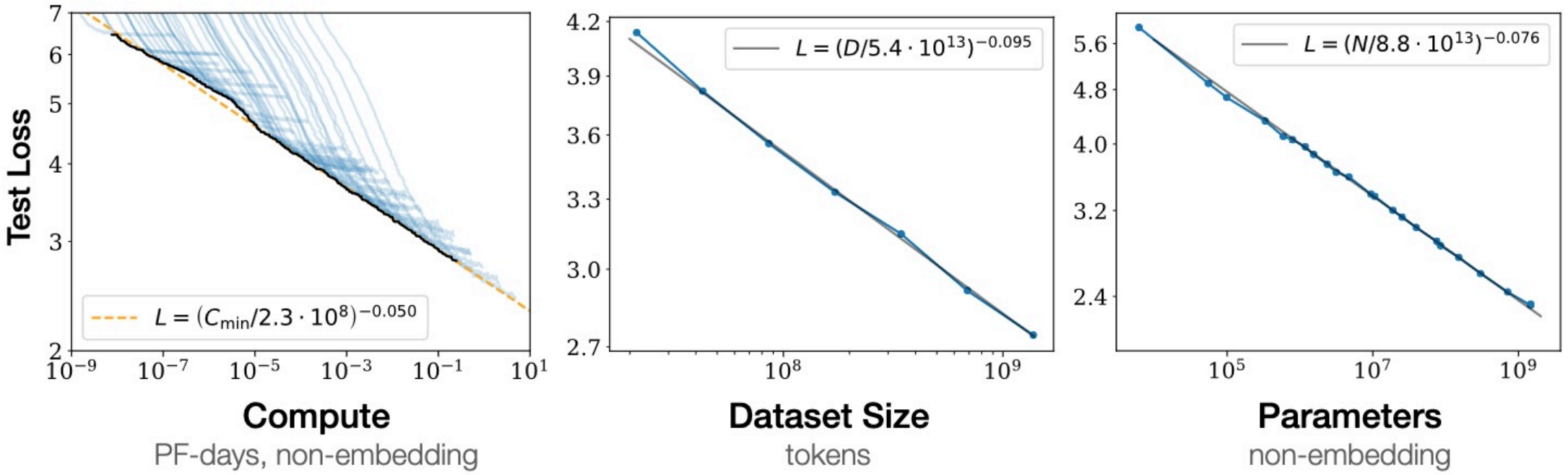


Image source: Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., ... & Amodei, D. (2020). Scaling laws for neural language models. arXiv preprint arXiv:2001.08361.

The Context Window

The **context window** (or **context length**) is the maximum number of tokens the model can process in a single forward pass — the maximum value of n.

Everything the model can "see" and reason about at inference time must fit within this window:

- The system prompt
- The conversation history
- The current user input
- Any retrieved documents

If the input exceeds the context window, some tokens must be truncated, summarized or retrieved separately — tokens outside the current window are not processed in that forward pass.

Context Window: Memory and Compute

The self-attention operation computes an $n \times n$ score matrix, so its memory and compute costs grow **quadratically** with context length n:

$$\text{Attention score matrix size} = n^2 \text{ elements per head}$$

A model with a **16k context window** can attend to twice as many tokens as an 8k model, but requires **four times** the memory for the attention computation.

Context Window: Practical Meaning

What 8k vs. 16k means in practice:

Task	8k context	16k context
Short-document Q&A	✅ Fits easily	✅ Fits easily
Long article summarization	⚠️ May truncate	✅ Usually fits
Full book processing	❌ Requires chunking	❌ Requires chunking
Multi-turn dialogue	⚠️ History lost quickly	✅ Longer history preserved

The supported context window is determined by the architecture, positional encoding and training setup. Learned absolute positional embeddings impose a hard table size. The usable context can be set **lower at runtime** (e.g. in inference software such as LM Studio) to reduce KV-cache memory consumption.

Summary: The Transformer Architecture

The full Transformer pipeline from raw text to output prediction:

1. **Tokenize** raw text → sequence of token IDs using BPE or similar subword algorithm
2. **Embed** each token ID → dense vector via the embedding matrix $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$
3. **Add positional encoding** → sinusoidal $PE(pos, i)$ added element-wise to each token embedding
4. **Encoder** (if present) → stack of bidirectional self-attention + FFN layers; produces key-value representations for the decoder
5. **Decoder** → stack of masked self-attention + cross-attention + FFN layers; generates output tokens autoregressively
6. **Output projection** → linear layer $\mathbf{W}_{\text{out}}$ + softmax → next-token probability distribution over $\mathcal{V}$

Summary: Why Transformers Work

Challenge	Transformer solution
Sequential bottleneck of RNNs	Parallel self-attention over all positions simultaneously
Vanishing gradients over long sequences	Direct attention connections between any two positions
Fixed-size context bottleneck in RNN seq2seq	Full encoder output accessible at every decoder step via cross-attention
Lack of positional information	Sinusoidal positional encoding $PE(pos, i)$ added to embeddings
Single attention pattern insufficient	Multi-head attention with H independent learned subspaces
Deep network training instability	Residual connections + layer normalization at every sublayer